ESGI Lyon
5ESGI AL, année 2026-2027

Clusterisation de conteneurs

Rapport des travaux dirigés

Docker, Docker Compose, Docker Swarm, Traefik, Portainer,
Dozzle, Authelia, Outline

Ahmed Yahya LETAIEF

Novembre 2026

Sommaire

Présentation **4**

TD 1 : Dockerisation de My Favorite Places **5**

Exercice 1 : Dockerisation du projet . . . 5

1.1 Tester le serveur avec yarn dev et Bruno . . . 5

1.2 Dockeriser le serveur . . . 6

1.2.a Le Dockerfile . . . 6

1.2.b Lancer la base et le serveur . . . 6

1.2.c Test avec Bruno . . . 7

1.3 Créer un compose.yml pour le serveur et la BDD . . . 8

1.4 Dockeriser le client et l'ajouter au compose.yml . . . 9

1.4.a Dockeriser le client . . . 9

1.4.b Ajouter le client au compose.yml . . . 11

1.5 Communication entre la BDD, le serveur et le front . . . 11

1.5.a Entre la BDD et le serveur . . . 12

1.5.b Entre le serveur et le front . . . 12

Exercice 2 : Démarrage, redémarrage et images légères . . . 12

2.1 Démarrer le serveur une fois la BDD prête . . . 12

2.2 Redémarrer automatiquement les services . . . 12

2.3 Optimiser les images avec les build stages . . . 13

2.3.a Serveur . . . 13

2.3.b Client . . . 13

2.3.c Résultat . . . 14

TD 2 : Cluster Docker Swarm **15**

Prérequis : images publiées sur un registry . . . 15

0.1 Publier les images avec GitHub Actions . . . 15

0.2 Vérifier les images sur GHCR . . . 15

Exercice 2 : Création du cluster Docker Swarm . . . 15

2.1 Créer et lancer les 4 conteneurs DinD . . . 15

2.2 Initialiser le cluster depuis le manager . . . 16

2.3 Faire rejoindre les nœuds comme workers . . . 17

2.4 Lister les nœuds depuis le manager . . . 17

Exercice 3 : Tests du cluster . . . 17

3.1 Créer la stack hello-world . . . 17

3.2 Copier la stack dans le manager . . . 18

3.3 Déployer la stack et observer les nœuds . . . 18

3.3.a Relancer le cluster après un redémarrage du PC . . . 18

3.3.b Déployer la stack et regarder chaque nœud. . . . 19
3.4 Faire varier la clause deploy . . . 20
3.4.a Sur le manager seulement (par rôle) . . . 20
3.4.b Sur tous les nœuds (mode global) . . . 21

TD 3 : Traefik et Portainer **23**

Exercice 1 : Déployer Traefik . . . 23
1.1 Relancer le cluster . . . 23
1.2 Déployer Traefik sur le manager . . . 23
1.3 Accéder à Traefik depuis le navigateur . . . 24
Exercice 2 : Déployer une autre application . . . 25
2.1 Tester l'application avec Docker Compose . . . 25
2.2 Déployer l'application sur le cluster Swarm . . . 26
2.3 Publier le vote et les résultats avec Traefik . . . 27
Exercice 3 : Déployer Portainer . . . 28
3.1 Déployer Portainer sur le cluster. . . . 28
3.2 Publier Portainer avec Traefik . . . 29
3.3 Ouvrir Portainer dans le navigateur . . . 29
Exercice 4 : Redéployer depuis Portainer . . . 30
4.1 Supprimer la stack depuis le manager . . . 30
4.2 Republier la stack depuis Portainer . . . 31
4.3 Vérifier l'accès depuis le navigateur . . . 32
4.4 Déployer l'API de MFP depuis Portainer . . . 32
4.4.a Création de la stack . . . 32
4.4.b Test avec Bruno . . . 32

TD 4 : Secrets et configs Docker Swarm **34**

Prérequis : relancer le cluster et passer en HTTPS . . . 34
0.1 Relancer le cluster . . . 34
0.2 Créer le certificat . . . 35
0.3 Le certificat dans le cluster : un secret et des configs . . . 35
0.4 Traefik en HTTPS . . . 36
0.5 Relancer Portainer et ajouter les adresses . . . 37
Exercice 1 : Déployer des logiciels . . . 37
1.1 Dozzle avec un utilisateur simple . . . 37
1.1.a Le fichier users.yml . . . 37
1.1.b Le secret, créé dans Portainer . . . 37
1.1.c La stack . . . 38
1.1.d Test dans le navigateur . . . 38
1.2 Authelia et connexion OpenID Connect . . . 38
1.2.a Les clés d'Authelia . . . 39
1.2.b Le compte dans un secret. . . . 39
1.2.c Le client OIDC de Dozzle . . . 39

1.2.d La configuration dans une config . . . 39
1.2.e Le déploiement . . . 40
1.2.f Brancher Dozzle sur Authelia . . . 41
1.2.g Rôles manquants et mise à jour de la config . . . 41
1.2.h Connexion à Dozzle par Authelia. . . . 42
1.3 Bonus : Outline . . . 43
1.3.a Les secrets d'Outline . . . 43
1.3.b Un nouveau client dans Authelia . . . 43
1.3.c La stack . . . 44
1.3.d Connexion à Outline . . . 45
Bilan. . . . 45

Présentation

Dépôt : `https://github.com/DARK-SHAD0W/Clusterisation-Conteneurs`

- `mfp/` : l'application My Favorite Places (client React, API Node.js/Express, base PostgreSQL) et le cluster Docker Swarm (`mfp/swarm/`) ;
- `voting-app/` : l'application de vote du TD 3.

Environnement de travail :

Outil	Version	Remarque
Docker Engine	29.7.2	dans WSL 2 (Ubuntu 24.04), sans Docker Desktop
Docker Compose	v5.5.0	plugin `docker compose`
Docker des nœuds du cluster	29.8.2	image `docker:dind`
Node.js / Yarn	22.23.2 / 1.22.22	dans WSL
Bruno	4.2.1	client API, sous Windows

Les commandes du cluster se lancent depuis le dossier `mfp/swarm/`, qui contient son `compose.yml`.

TD 1 : Dockerisation de My Favorite Places

My Favorite Places est une petite application avec un client React, une API Node.js/Express et une base PostgreSQL. Dans ce TD, je l'ai mise dans des conteneurs, puis je l'ai lancée avec Docker Compose.

Exercice 1 : Dockerisation du projet

1.1 Tester le serveur avec yarn dev et Bruno

Le serveur refuse de démarrer tant qu'il ne trouve pas PostgreSQL. J'ai donc d'abord lancé une base dans un conteneur, avec le mot de passe écrit dans `server/src/datasource.ts`.

Commandes

```
docker run -d --name mfp-db \
  -e POSTGRES_PASSWORD=supersecret \
  -p 5432:5432 \
  postgres:17
docker ps

cd server
yarn install
yarn dev
```

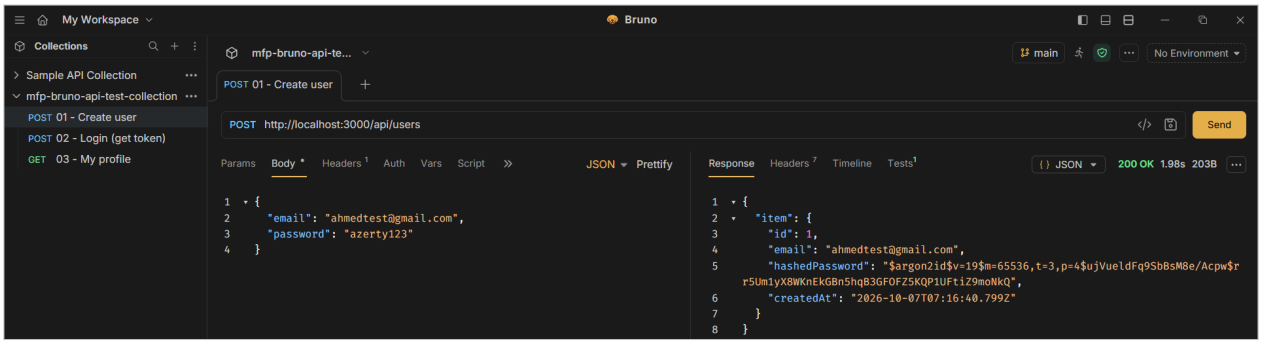


Figure 1.1. L'utilisateur de test est créé, avec un mot de passe haché en Argon2.

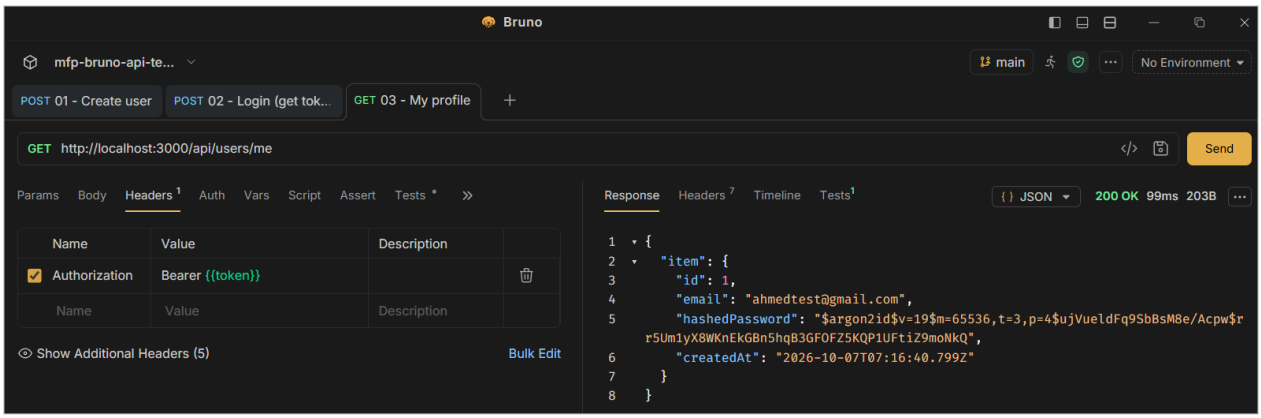


Figure 1.2. Avec le token reçu à la connexion, `/api/users/me` renvoie bien mon profil.

1.2 Dockeriser le serveur

1.2.a Le Dockerfile

server/Dockerfile

```
FROM node:24-alpine3.24
WORKDIR /app
COPY package.json yarn.lock ./
RUN yarn install --frozen-lockfile
COPY tsconfig.json ./
COPY src src
RUN yarn build
EXPOSE 3000
CMD ["yarn", "start"]
```

Je copie `package.json` et `yarn.lock` avant le code pour que Docker garde `yarn install` en cache. Sur les captures, l'image est encore `node:22`, remplacée ensuite par `node:24-alpine3.24`.

1.2.b Lancer la base et le serveur

Dans un conteneur, `localhost` désigne le conteneur lui-même, donc le serveur ne pouvait plus trouver la base. Je lui ai fait lire l'adresse de la base dans des variables d'environnement.

server/src/datasource.ts

```
host: process.env.DB_HOST || "localhost",
username: process.env.DB_USER || "postgres",
password: process.env.DB_PASSWORD || "supersecret",
database: process.env.DB_NAME || "postgres",
```

J'ai ensuite mis les deux conteneurs sur un même réseau, sinon le nom `mfp-db` n'est pas résolu.

Commandes

```
docker network create mfp-net
docker run -d --name mfp-db --network mfp-net \
  -e POSTGRES_PASSWORD=supersecret \
  postgres:17
docker build -t mfp-server ./server
docker run -d --name mfp-server --network mfp-net \
  -e DB_HOST=mfp-db \
  -p 3000:3000 \
  mfp-server
```

Problème rencontré : `address already in use`, parce que le `yarn dev` de 1.1 tournait encore.
Solution : J'ai arrêté `yarn dev`, supprimé le conteneur raté et relancé le serveur.

Commandes

```
docker rm -f mfp-server
docker run -d --name mfp-server --network mfp-net \
  -e DB_HOST=mfp-db \
  -p 3000:3000 \
  mfp-server
docker ps
docker logs mfp-server
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$ docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS         PORTS
                                NAMES
c1acc99cd601   mfp-server    "docker-entrypoint.s…"   2 minutes ago   Up 2 minutes   0.0.0.
0:3000->3000/tcp, [::]:3000->3000/tcp   mfp-server
c0c978bba581   postgres:17   "docker-entrypoint.s…"   9 minutes ago   Up 9 minutes   5432/t
cp                              mfp-db
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$ docker logs mfp-server
yarn run v1.22.22
$ node dist/index.js
⏳ starting server!
query: SELECT version()
query: SELECT * FROM current_schema()
query: START TRANSACTION
query: SELECT * FROM current_schema()
query: SELECT * FROM current_database()
query: SELECT "table_schema", "table_name", obj_description(('"' || "table_schema" || '"."'
 || "table_name" || '"')::regclass, 'pg_class') AS table_comment FROM "information_schema".
"tables" WHERE ("table_schema" = 'public' AND "table_name" = 'address') OR ("table_schema"
= 'public' AND "table_name" = 'user')
query: SELECT * FROM "information_schema"."tables" WHERE "table_schema" = 'public' AND "tab
le_name" = 'typeorm_metadata'
query: CREATE TABLE "address" ("id" SERIAL NOT NULL, "name" character varying NOT NULL, "de
scription" character varying, "lng" double precision NOT NULL, "lat" double precision NOT N
ULL, "createdAt" TIMESTAMP NOT NULL DEFAULT now(), "userId" integer, CONSTRAINT "PK_d92de1f
82754668b5f5f5dd4fd5" PRIMARY KEY ("id"))
query: CREATE TABLE "user" ("id" SERIAL NOT NULL, "email" character varying NOT NULL, "hash
edPassword" character varying NOT NULL, "createdAt" TIMESTAMP NOT NULL DEFAULT now(), CONST
RAINT "PK_cace4a159ff9f2512dd42373760" PRIMARY KEY ("id"))
query: ALTER TABLE "address" ADD CONSTRAINT "FK_d25f1ea79e282cc8a42bd616aa3" FOREIGN KEY ("
userId") REFERENCES "user"("id") ON DELETE NO ACTION ON UPDATE NO ACTION
query: COMMIT
✅ connected to PgSQL db
🚀 server started on port 3000
query: START TRANSACTION
query: INSERT INTO "user"("email", "hashedPassword", "createdAt") VALUES ($1, $2, DEFAULT)
RETURNING "id", "createdAt" -- PARAMETERS: ["ahmedtest@gmail.com","$argon2id$v=19$m=65536,t
=3,p=4$QederZF6DNgzDmpXATUdJw$ZXH3ixyadbYjLkJdHMXKkZzYNbVv145xfwJgaaiNssw"]
query: COMMIT
query: SELECT "User"."id" AS "User_id", "User"."email" AS "User_email", "User"."hashedPassw
ord" AS "User_hashedPassword", "User"."createdAt" AS "User_createdAt" FROM "user" "User" WH
ERE ("User"."email" = $1) LIMIT 1 -- PARAMETERS: ["ahmedtest@gmail.com"]
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$
```

Figure 1.3. Cette fois dans Docker, le serveur crée ses tables dans `mfp-db` et démarre.

1.2.c Test avec Bruno

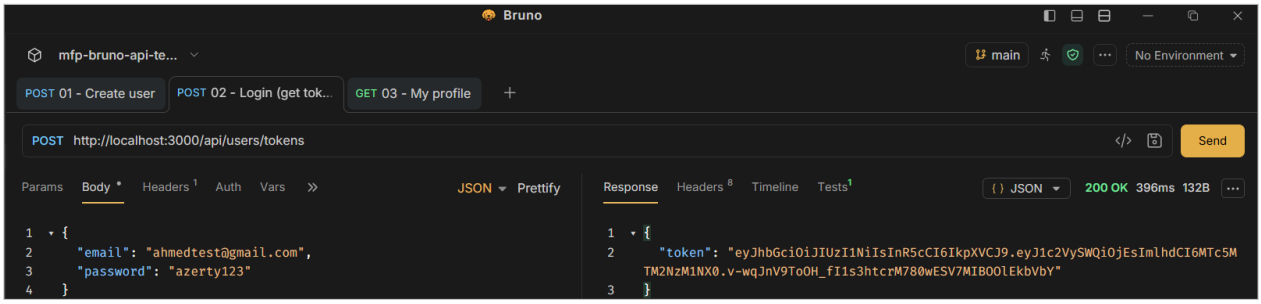


Figure 1.4. Après avoir recréé l'utilisateur dans la base neuve, la connexion renvoie bien un token.

1.3 Créer un compose.yml pour le serveur et la BDD

Compose crée le réseau tout seul et chaque service y est joignable par son nom, ce qui remplace mes `docker network` et `docker run`.

compose.yml

```yaml
services:
  db:
    image: postgres:17
    # ...
    volumes:
      - db-data:/var/lib/postgresql/data
    # ...
  server:
    build: ./server
    # ...
    ports:
      - "3000:3000"
```

En pratique, j'avais déjà dockerisé le client (question 1.4) quand j'ai écrit ce fichier, et mes conteneurs lancés à la main gardaient les ports 3000 et 5173. Je les ai supprimés avant le premier `docker compose up`.

Commandes

```
docker rm -f mfp-client mfp-server mfp-db
docker network rm mfp-net
docker compose up -d --build
docker compose ps
docker compose logs server
```

Problème rencontré : Avec un simple `depends_on`, le serveur s'arrêtait sur `ECONNREFUSED`.
Solution : J'ai ajouté un `healthcheck` sur la base, détaillé en 2.1.

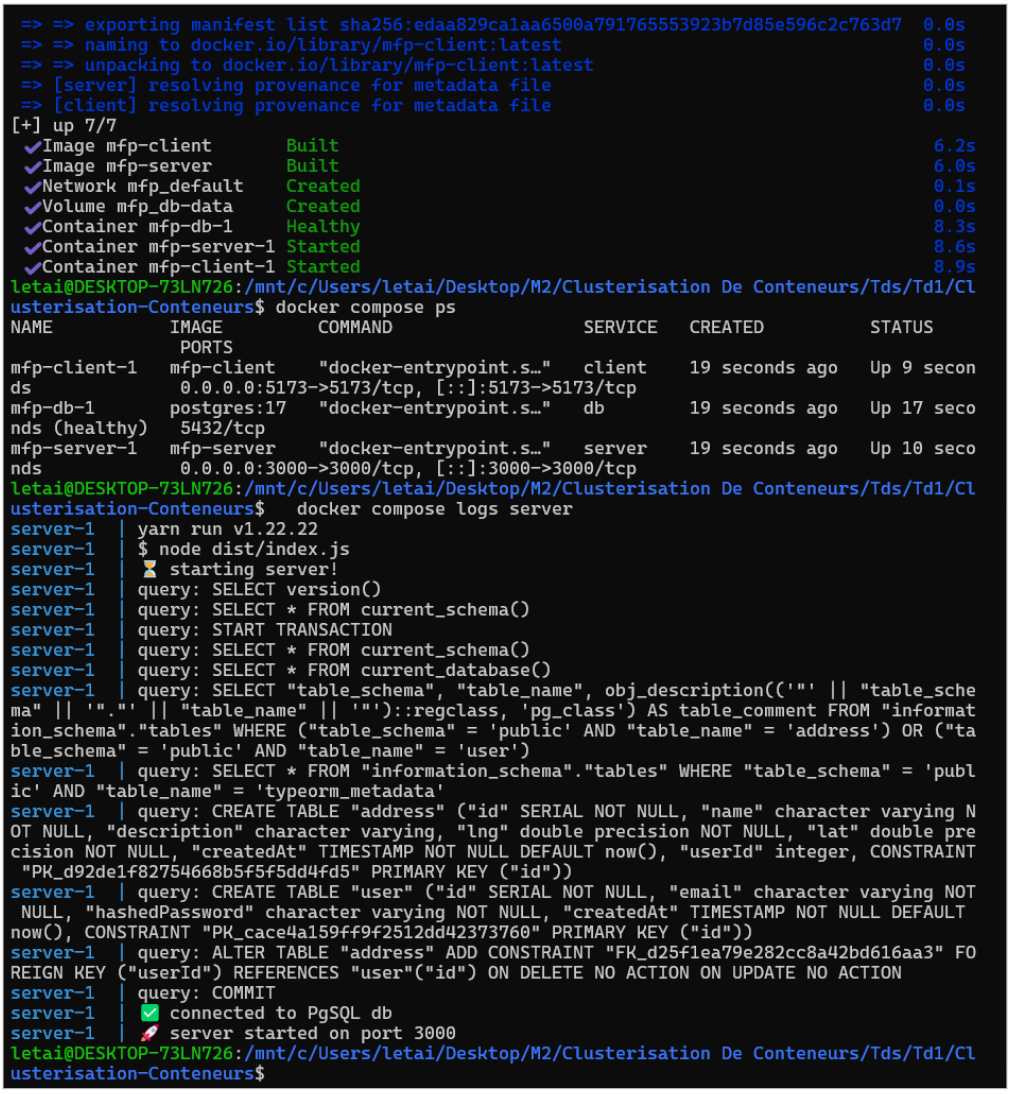


Figure 1.5. Avec le healthcheck, le serveur attend que `mfp-db-1` soit `Healthy` et se connecte du premier coup.

Dans `datasource.ts`, j'ai fini par écrire le nom `db` en dur à la place des variables, puisque c'est toujours le nom de la base avec Compose. Le revers, c'est que le serveur ne marche plus en dehors de Compose.

1.4 Dockeriser le client et l'ajouter au compose.yml

1.4.a Dockeriser le client

client/Dockerfile

```
FROM node:24-alpine3.24
WORKDIR /app
COPY package*.json yarn.lock ./
RUN yarn install --frozen-lockfile
# ...
EXPOSE 5173
CMD ["yarn", "dev", "--host"]
```

Sans `--host`, Vite n'accepterait que les connexions venant de l'intérieur du conteneur.

Il fallait aussi changer l'adresse vers laquelle Vite fait suivre les appels `/api`, parce que depuis ce

conteneur, `localhost:3000` ne mène plus au serveur. Je l'ai mise dans une variable, comme pour la base.

client/vite.config.ts

```
target: process.env.API_URL || "http://localhost:3000",
```

Commandes

```
docker build -t mfp-client ./client
docker run -d --name mfp-client --network mfp-net \
  -e API_URL=http://mfp-server:3000 \
  -p 5173:5173 \
  mfp-client
docker ps
docker logs mfp-client
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$ docker build -t mfp-client ./client
[+] Building 186.1s (12/12) FINISHED                                    docker:default
 => [internal] load build definition from Dockerfile                             0.3s
 => => transferring dockerfile: 465B                                             0.2s
 => [internal] load metadata for docker.io/library/node:22                       0.7s
 => [internal] load .dockerignore                                                0.2s
 => => transferring context: 58B                                                 0.1s
 => [1/7] FROM docker.io/library/node:22@sha256:0e5f906573693feaa1e21057ebdcfdb5bd50  0.1s
 => => resolve docker.io/library/node:22@sha256:0e5f906573693feaa1e21057ebdcfdb5bd50  0.1s
 => [internal] load build context                                                0.5s
 => => transferring context: 108.53kB                                            0.4s
 => CACHED [2/7] WORKDIR /app                                                    0.0s
 => [3/7] COPY package.json yarn.lock ./                                         0.1s
 => [4/7] RUN yarn install --frozen-lockfile                                    54.8s
 => [5/7] COPY index.html vite.config.ts tsconfig.json tsconfig.app.json tsconfig.no  0.8s
 => [6/7] COPY public public                                                     0.2s
 => [7/7] COPY src src                                                           0.2s
 => exporting to image                                                         127.9s
 => => exporting layers                                                         98.6s
 => => exporting manifest sha256:dd7d4565eadb69e779f069a2ce1a040d03fde7e3534f2820eed  0.0s
 => => exporting config sha256:3070b6bb523c3d8b165f6206056fa31c565b1dcaa7b42361f2b75  0.0s
 => => exporting attestation manifest sha256:14cad0cf8fc84ae057de56db50d52c0ff95ac8e  0.0s
 => => exporting manifest list sha256:bc628b83aa66138ac7a53df0e230f8e63d25186bb23e01  0.0s
 => => naming to docker.io/library/mfp-client:latest                             0.0s
 => => unpacking to docker.io/library/mfp-client:latest                         29.0s
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$ docker run -d --name mfp-client --network mfp-net \
  -e API_URL=http://mfp-server:3000 \
  -p 5173:5173 \
  mfp-client
846fcb38043009ec7f4d7a70d92cf27f9d033577d0055c71cda60564b4b47207
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$ docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS          PORT
S                                       NAMES
846fcb380430   mfp-client    "docker-entrypoint.s…"   19 seconds ago   Up 17 seconds   0.0.
0.0:5173->5173/tcp, [::]:5173->5173/tcp   mfp-client
c1acc99cd601   mfp-server    "docker-entrypoint.s…"   13 minutes ago   Up 13 minutes   0.0.
0.0:3000->3000/tcp, [::]:3000->3000/tcp   mfp-server
c0c978bba581   postgres:17   "docker-entrypoint.s…"   20 minutes ago   Up 20 minutes   5432
/tcp                                    mfp-db
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$ docker logs mfp-client
yarn run v1.22.22
$ vite --host

  VITE v7.3.1  ready in 798 ms

  ➜ Local:   http://localhost:5173/
  ➜ Network: http://172.18.0.4:5173/
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs$
```

Figure 1.6. Les trois conteneurs tournent, et Vite écoute sur le port 5173.

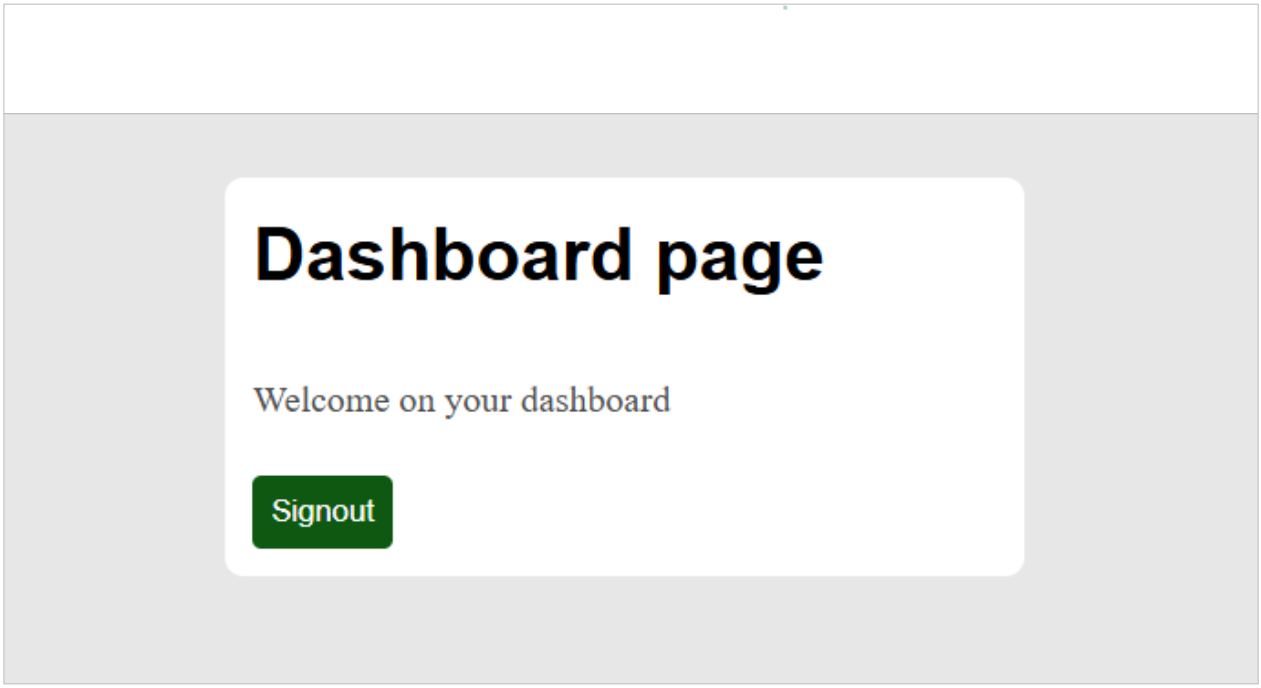


Figure 1.7. Je me connecte depuis le client dans Docker, donc ses appels `/api` arrivent bien au serveur.

1.4.b Ajouter le client au compose.yml

compose.yml

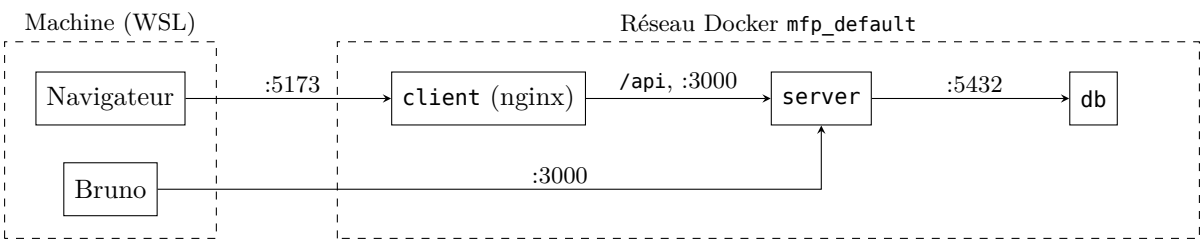
```yaml
client:
  build: ./client
  # ...
  ports:
    - "5173:5173"
  depends_on:
    - server
```

Le client a donc démarré avec le `docker compose up` de 1.3. Comme le volume était neuf, j'ai recréé l'utilisateur avec Bruno, puis la connexion sur le port 5173 a marché comme avant. Un `docker compose down` arrête tout sans effacer le volume `db-data`, donc les données restent.

Commandes

```
docker compose down
```

1.5 Communication entre la BDD, le serveur et le front



1.5.a Entre la BDD et le serveur

Ils sont sur le même réseau, `mfp_default`, que Compose crée tout seul. Le serveur appelle la base par son nom de service, `db`, sur le port 5432, et c'est le DNS interne de Docker qui donne l'adresse IP. Je publie quand même le port 5432, mais seulement pour regarder la base depuis ma machine.

1.5.b Entre le serveur et le front

Le front ne parle pas directement au serveur, parce que le code React tourne dans le navigateur, en dehors du réseau Docker. Il appelle `/api` sur l'adresse du site, `localhost:5173`, et c'est le conteneur `client` qui fait suivre au service `server` (Vite au début, nginx ensuite).

Exercice 2 : Démarrage, redémarrage et images légères

2.1 Démarrer le serveur une fois la BDD prête

Je l'avais déjà fait en 1.3, après le plantage du serveur au premier `docker compose up`. Toutes les 2 secondes, `pg_isready` teste la base, et le serveur attend qu'elle soit `healthy` pour démarrer.

compose.yml

```yaml
services:
  db:
    # ...
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 2s
      # ...
  server:
    # ...
    depends_on:
      db:
        condition: service_healthy
```

2.2 Redémarrer automatiquement les services

Avec `unless-stopped`, Docker relance un service qui plante, mais pas un service que j'ai arrêté moi-même. J'ai mis cette ligne sur les trois services.

compose.yml

```yaml
restart: unless-stopped
```

Commandes

```
docker compose exec server pkill -f "node dist/index.js"
docker inspect mfp-server-1 --format 'status={{.State.Status}} restarts={{.RestartCount}}'
# status=running restarts=1

docker compose stop server
docker inspect mfp-server-1 --format 'status={{.State.Status}} restarts={{.RestartCount}}'
# status=exited restarts=1   (arrêt manuel : pas de redémarrage)

docker compose start server
```

2.3 Optimiser les images avec les build stages

J'ai découpé chaque Dockerfile en trois étapes. Seule la dernière sert d'image finale, donc les outils de build et le code source n'y sont plus.

2.3.a Serveur

server/Dockerfile

```
FROM node:24-alpine3.24 AS deps
# ...
FROM node:24-alpine3.24 AS builder
COPY --from=deps /app/node_modules ./node_modules
# ...
FROM node:24-alpine3.24 AS main
# ...
COPY --from=builder /app/dist ./dist
COPY package.json yarn.lock ./
RUN yarn install --frozen-lockfile --production && yarn cache clean
# ...
CMD ["yarn", "start"]
```

2.3.b Client

Les deux premières étapes sont les mêmes que pour le serveur. Une fois buildé, le client n'est plus qu'un dossier de fichiers statiques, alors je les fais servir par nginx au lieu de Node. Le `try_files` sert aux routes React comme `/dashboard`, qui n'existent pas en tant que fichiers.

client/Dockerfile

```
# ...
FROM nginx:1.29-alpine AS main
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 5173
```

client/nginx.conf

```
server {
    listen 5173;
    # ...
    location / {
        try_files $uri $uri/ /index.html;
    }
    location /api/ {
        proxy_pass http://server:3000;
    }
}
```

2.3.c Résultat

Commandes

```
docker compose up -d --build
docker images
```

Image	Avant	Après
`mfp-server`	477 Mo	337 Mo
`mfp-client`	1,49 Go	93 Mo

L'image du serveur a perdu environ 140 Mo (de 477 à 337 Mo), donc plus que les 100 Mo environ demandés. Le client fond encore plus, parce que son image finale ne contient plus que nginx et les fichiers buildés.

TD 2 : Cluster Docker Swarm

Dans ce TD, j'ai monté un petit cluster Docker Swarm sur mon PC. Chaque machine du cluster est en fait un conteneur Docker-in-Docker, avec un manager et trois nœuds.

Prérequis : images publiées sur un registry

0.1 Publier les images avec GitHub Actions

Swarm ne sait pas construire les images, il faut donc qu'elles soient déjà sur un registry. J'ai écrit deux workflows qui les construisent et les publient sur GHCR à chaque push sur `main`.

.github/workflows/build.server.yml

```yaml
env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}/server
```

0.2 Vérifier les images sur GHCR

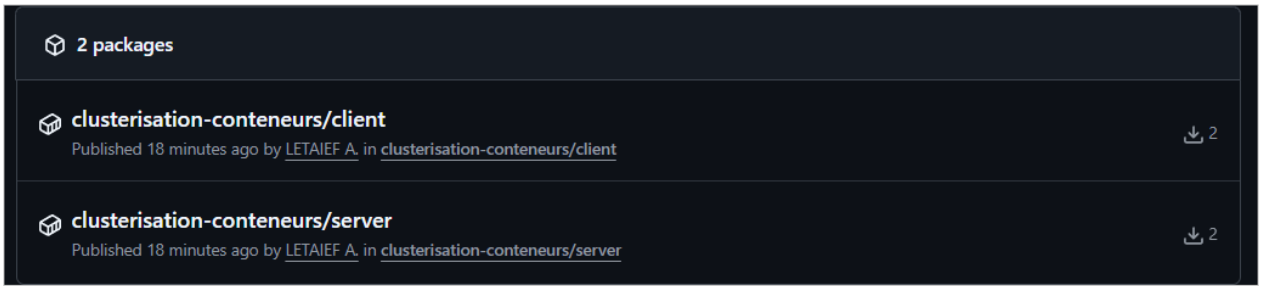


Figure 2.1. Les deux images sont publiques, donc les nœuds peuvent les télécharger sans se connecter.

Exercice 2 : Création du cluster Docker Swarm

2.1 Créer et lancer les 4 conteneurs DinD

Les trois nœuds sont déclarés comme le manager. Sans `privileged: true`, Docker ne peut pas tourner dans le conteneur.

swarm/compose.yml

```yaml
services:
  manager:
    image: docker:dind
    hostname: manager
    privileged: true
  # ...
```

Commandes

```
cd swarm
docker compose up -d
docker ps
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs/swarm$  docker compose up -d
[+] up 22/22
 ✔Image docker:dind         Pulled                                                    29.5s
 ✔Network swarm_default     Created                                                    0.1s
 ✔Container swarm-node3-1   Started                                                    1.4s
 ✔Container swarm-manager-1 Started                                                    1.5s
 ✔Container swarm-node1-1   Started                                                    1.4s
 ✔Container swarm-node2-1   Started                                                    1.5s
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
usterisation-Conteneurs/swarm$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED             STATUS
      PORTS                                                 NAMES
8db993eaa83c   docker:dind    "dockerd-entrypoint.…"   24 seconds ago      Up 22 seconds
      2375-2376/tcp                                         swarm-manager-1
12a3a3bf1b09   docker:dind    "dockerd-entrypoint.…"   24 seconds ago      Up 22 seconds
      2375-2376/tcp                                         swarm-node2-1
aacec484287b   docker:dind    "dockerd-entrypoint.…"   24 seconds ago      Up 22 seconds
      2375-2376/tcp                                         swarm-node3-1
57aa55d79f91   docker:dind    "dockerd-entrypoint.…"   24 seconds ago      Up 22 seconds
      2375-2376/tcp                                         swarm-node1-1
11ed06c1a8a4   mfp-client     "/docker-entrypoint.…"   About an hour ago   Up About an hour
      80/tcp, 0.0.0.0:5173->5173/tcp, [::]:5173->5173/tcp   mfp-client-1
f6b1980ab4cb   c9ab01022362   "docker-entrypoint.s…"   2 hours ago         Up About an hour
      0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp           mfp-server-1
4c4030c6d31d   postgres:17    "docker-entrypoint.s…"   2 hours ago         Up 2 hours (heal
thy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp           mfp-db-1
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Cl
```

Figure 2.2. Les quatre conteneurs DinD tournent à côté de ceux du TD 1.

Commandes

```
docker compose exec manager docker ps
docker compose exec node1 docker ps
```

Dans les deux conteneurs, `docker ps` répond avec une liste vide. Docker marche donc bien à l'intérieur, et chaque machine a le sien.

2.2 Initialiser le cluster depuis le manager

Commandes

```
docker compose exec manager docker swarm init
```

Le manager devient le leader du cluster et affiche le token `SWMTKN-1-...` dont les workers ont besoin pour rejoindre le cluster.

2.3 Faire rejoindre les nœuds comme workers

Plutôt que de recopier le token à la main, je l'ai mis dans une variable. Les nœuds rejoignent le manager par son nom de service `manager:2377`, et pas par son adresse IP.

Commandes

```
TOKEN=$(docker compose exec manager docker swarm join-token -q worker)
docker compose exec node1 docker swarm join --token $TOKEN manager:2377
docker compose exec node2 docker swarm join --token $TOKEN manager:2377
docker compose exec node3 docker swarm join --token $TOKEN manager:2377
```

2.4 Lister les nœuds depuis le manager

Commandes

```
docker compose exec manager ash
docker node ls
exit
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusteris
ation-Conteneurs/swarm$  docker compose exec manager ash
/ #  docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
ut2j0ctz6094h5y236qwnbii5 *   manager    Ready     Active         Leader           29.8.2
gkqoimullj55ae70vubvjpq3t     node1      Ready     Active                          29.8.2
htlyy0s49t7h787sznf9syr0h     node2      Ready     Active                          29.8.2
tih4g88m7eipgra6a02nwvi17     node3      Ready     Active                          29.8.2
/ # exit
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusteris
ation-Conteneurs/swarm$
```

Figure 2.3. Le cluster compte bien 4 nœuds `Ready`, et le manager est `Leader`.

Exercice 3 : Tests du cluster

3.1 Créer la stack hello-world

swarm/hello-world.compose.yml

```
services:
  hello:
    image: hello-world
    deploy:
      replicas: 2
```

3.2 Copier la stack dans le manager

J'ai ajouté un volume anonyme sur `/home/manager` pour ne pas perdre la stack si le conteneur est recréé.

swarm/compose.yml

```yaml
manager:
  # ...
  volumes:
    - /home/manager
```

Commandes

```
docker compose up -d manager
docker compose exec manager docker node ls
docker compose exec manager mkdir -p /home/manager
docker compose cp hello-world.compose.yml manager:/home/manager/
docker compose exec manager ls /home/manager
```

Même recréé, le manager voyait toujours ses 4 nœuds `Ready`.

3.3 Déployer la stack et observer les nœuds

3.3.a Relancer le cluster après un redémarrage du PC

Commandes

```
docker compose start
docker compose exec manager docker node ls
```

Problème rencontré : Les nœuds restaient `Down`, car Swarm avait gardé l'ancienne IP du manager.
Solution : J'ai fait quitter le cluster à toutes les machines et je l'ai recréé.

Commandes

```
docker compose exec node1 docker swarm leave --force
docker compose exec node2 docker swarm leave --force
docker compose exec node3 docker swarm leave --force
docker compose exec manager docker swarm leave --force

docker compose exec manager docker swarm init
TOKEN=$(docker compose exec manager docker swarm join-token -q worker)
docker compose exec node1 docker swarm join --token $TOKEN manager:2377
docker compose exec node2 docker swarm join --token $TOKEN manager:2377
docker compose exec node3 docker swarm join --token $TOKEN manager:2377

docker compose exec manager docker node ls
```

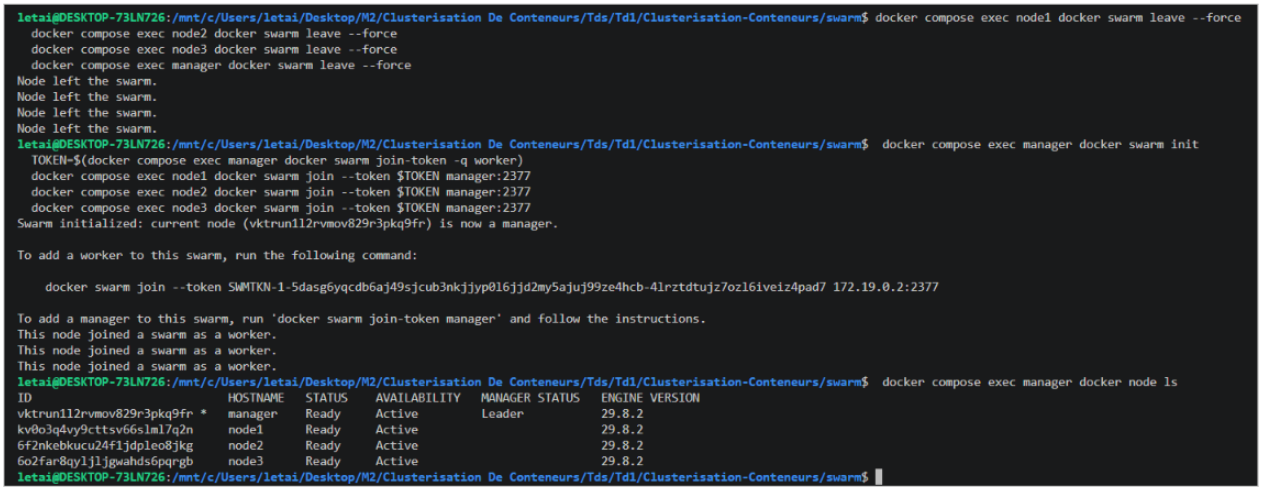


Figure 2.4. Après la recréation, les 4 nœuds sont de nouveau `Ready`.

3.3.b Déployer la stack et regarder chaque nœud

Commandes

```
docker compose exec manager docker stack deploy -c /home/manager/hello-world.compose.yml
↪  hello
docker compose exec manager docker service ls
docker compose exec manager docker service ps hello_hello
```

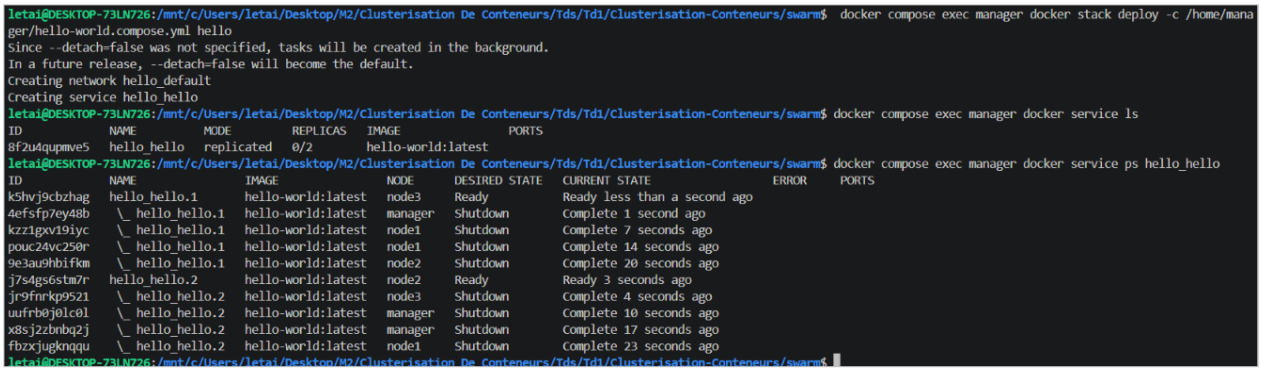


Figure 2.5. Le service `hello_hello` reste à `0/2` et ses copies passent d'un nœud à l'autre.

Pour comparer, j'ai lancé un service nginx qui reste allumé, et lui affiche bien `3/3`.

Commandes

```
docker compose exec manager docker service create --name web --replicas 3 nginx:alpine
docker compose exec manager docker service ls
docker compose exec manager docker service rm web

docker compose exec manager docker ps -a
docker compose exec node1 docker ps -a
docker compose exec node2 docker ps -a
docker compose exec node3 docker ps -a
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$  docker compose exec manager do
cker ps -a
CONTAINER ID   IMAGE              COMMAND    CREATED          STATUS                      PORTS     NAMES
f355638693eb   hello-world:latest   "/hello"   9 seconds ago    Exited (0) 2 seconds ago            hello_hello.1.xsi3ohviq3j43lx5xuadtdrvw
f24aef73124b   hello-world:latest   "/hello"   16 seconds ago   Exited (0) 10 seconds ago           hello_hello.1.7sigjlu29gbypa3hod40cx39m
bfbf0de6f4ad   hello-world:latest   "/hello"   24 seconds ago   Exited (0) 17 seconds ago           hello_hello.2.y2xt2fd35e37cbadcod13i7pa
0833d06d9755   hello-world:latest   "/hello"   31 seconds ago   Exited (0) 24 seconds ago           hello_hello.2.5a31wbqdf4gqphyevylmc2vky
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec node1 docke
r ps -a
CONTAINER ID   IMAGE              COMMAND    CREATED          STATUS                      PORTS     NAMES
b12bf34dff67   hello-world:latest   "/hello"   2 seconds ago    Created                             hello_hello.2.noviml5uf5s40265yxrk5806n
07d1bfb7b503   hello-world:latest   "/hello"   27 seconds ago   Exited (0) 22 seconds ago           hello_hello.2.6uho387zyzllzoeie4jiux84v
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec node2 docke
r ps -a
CONTAINER ID   IMAGE              COMMAND    CREATED          STATUS                      PORTS     NAMES
939973e82e41   hello-world:latest   "/hello"   22 seconds ago   Exited (0) 16 seconds ago           hello_hello.1.0fmw0lejau2izpjzruozpsw7v
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec node3 docke
r ps -a
CONTAINER ID   IMAGE              COMMAND    CREATED          STATUS                      PORTS     NAMES
ada9e02e6213   hello-world:latest   "/hello"   23 seconds ago   Exited (0) 17 seconds ago           hello_hello.1.xg3wkzwk1r2udwnx33bxxaqxb
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
```

Figure 2.6. Chaque machine a ses propres copies de `hello-world`, le manager aussi.

J'en déduis que Swarm répartit les copies sur toutes les machines, manager compris, et que chaque machine ne voit que ses conteneurs. Comme `hello-world` s'arrête juste après son message, Swarm en relance sans arrêt pour garder ses 2 copies, d'où le `0/2` et tous ces conteneurs `Exited (0)`. C'est aussi pour ça que j'ai dû ajouter `-a`, sinon `docker ps` n'affichait presque rien.

3.4 Faire varier la clause deploy

3.4.a Sur le manager seulement (par rôle)

swarm/hello-world-manager.compose.yml

```yaml
# ...
deploy:
  replicas: 2
  placement:
    constraints:
      - node.role == manager
```

Commandes

```
docker compose exec manager docker stack rm hello
docker compose cp hello-world-manager.compose.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c
↪  /home/manager/hello-world-manager.compose.yml hello
docker compose exec manager docker service ps hello_hello
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
docker compose exec manager docker stack rm hello
Removing service hello_hello
Removing network hello_default
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
  docker compose cp hello-world-manager.compose.yml manager:/home/manager/
[+] copy 1/1
 ✓ swarm-manager-1 Copied hello-world-manager.compose.yml to swarm-manager-1:/home/manager/                    0.1s
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
 docker compose exec manager docker stack deploy -c /home/manager/hello-world-manager.compose.yml hello
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network hello_default
Creating service hello_hello
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
 docker compose exec manager docker service ps hello_hello
ID             NAME                IMAGE                NODE      DESIRED STATE   CURRENT STATE                ERROR
 PORTS
zfjhmrzuo4fz   hello_hello.1       hello-world:latest   manager   Ready           Ready less than a second ago

0z7vmg3rvvnu    \_ hello_hello.1   hello-world:latest   manager   Shutdown        Complete 1 second ago

0s8l81lgrk1a    \_ hello_hello.1   hello-world:latest   manager   Shutdown        Complete 7 seconds ago

qkg5g80puhls   hello_hello.2       hello-world:latest   manager   Ready           Ready less than a second ago

lod0t1rgd1ya    \_ hello_hello.2   hello-world:latest   manager   Shutdown        Complete 1 second ago

vq2ejxa6qsuo    \_ hello_hello.2   hello-world:latest   manager   Shutdown        Complete 7 seconds ago

letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
```

Figure 2.7. Avec la contrainte sur le rôle, toutes les copies tournent sur le manager.

3.4.b Sur tous les nœuds (mode global)

swarm/hello-world-global.compose.yml

```yaml
# ...
deploy:
  mode: global
```

Commandes

```
docker compose exec manager docker stack rm hello
docker compose cp hello-world-global.compose.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c
↪  /home/manager/hello-world-global.compose.yml hello
docker compose exec manager docker service ls
docker compose exec manager docker service ps hello_hello
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$  docker compose exec manager do
cker stack rm hello
Removing service hello_hello
Removing network hello_default
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$  docker compose cp hello-world-
global.compose.yml manager:/home/manager/
[+] copy 1/1
 ✓ swarm-manager-1 Copied hello-world-global.compose.yml to swarm-manager-1:/home/manager/                                                    0.1s
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager doc
ker stack deploy -c /home/manager/hello-world-global.compose.yml hello
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network hello_default
Creating service hello_hello
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$  docker compose exec manager do
cker service ls
ID             NAME          MODE      REPLICAS   IMAGE               PORTS
rno40lm6b9zl   hello_hello   global    0/0        hello-world:latest
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$  docker compose exec manager do
cker service ps hello_hello
ID             NAME                                     IMAGE               NODE      DESIRED STATE   CURRENT STATE                ERROR     PORTS
p16kq27gcitt   hello_hello.6f2nkebkucu24f1jdpleo8jkg     hello-world:latest   node2     Ready           Ready less than a second ago
spzm3npdcc0n    \_ hello_hello.6f2nkebkucu24f1jdpleo8jkg   hello-world:latest   node2     Shutdown        Complete 1 second ago
fbfe974s5a7d    \_ hello_hello.6f2nkebkucu24f1jdpleo8jkg   hello-world:latest   node2     Shutdown        Complete 9 seconds ago
kfi1wj4o3psc    \_ hello_hello.6f2nkebkucu24f1jdpleo8jkg   hello-world:latest   node2     Shutdown        Complete 15 seconds ago
ozlzlcgou9mc    \_ hello_hello.6f2nkebkucu24f1jdpleo8jkg   hello-world:latest   node2     Shutdown        Complete 22 seconds ago
74fck10ko4cn   hello_hello.6o2far8qyljljgwahds6pqrgb     hello-world:latest   node3     Ready           Ready less than a second ago
wgp9hf0ivajv    \_ hello_hello.6o2far8qyljljgwahds6pqrgb   hello-world:latest   node3     Shutdown        Complete 1 second ago
3274m4l4cf1d    \_ hello_hello.6o2far8qyljljgwahds6pqrgb   hello-world:latest   node3     Shutdown        Complete 9 seconds ago
c8lp7t71y9x8    \_ hello_hello.6o2far8qyljljgwahds6pqrgb   hello-world:latest   node3     Shutdown        Complete 15 seconds ago
68a30dpoer2i    \_ hello_hello.6o2far8qyljljgwahds6pqrgb   hello-world:latest   node3     Shutdown        Complete 22 seconds ago
r7cz5gaxghs8   hello_hello.kv0o3q4vy9cttsv66slml7q2n     hello-world:latest   node1     Ready           Ready 3 seconds ago
uipommkun2bz    \_ hello_hello.kv0o3q4vy9cttsv66slml7q2n   hello-world:latest   node1     Shutdown        Complete 4 seconds ago
iuw7w43mt34s    \_ hello_hello.kv0o3q4vy9cttsv66slml7q2n   hello-world:latest   node1     Shutdown        Complete 11 seconds ago
md9jqa42g20l    \_ hello_hello.kv0o3q4vy9cttsv66slml7q2n   hello-world:latest   node1     Shutdown        Complete 17 seconds ago
ntjsnf5aqd2t    \_ hello_hello.kv0o3q4vy9cttsv66slml7q2n   hello-world:latest   node1     Shutdown        Complete 23 seconds ago
vrwzcp2e75xy   hello_hello.vktrun1l2rvmov829r3pkq9fr     hello-world:latest   manager   Ready           Ready less than a second ago
t9dwf3qketly    \_ hello_hello.vktrun1l2rvmov829r3pkq9fr   hello-world:latest   manager   Shutdown        Complete 2 seconds ago
ok3tbu0hd9zo    \_ hello_hello.vktrun1l2rvmov829r3pkq9fr   hello-world:latest   manager   Shutdown        Complete 9 seconds ago
rtsx0d3rbglq    \_ hello_hello.vktrun1l2rvmov829r3pkq9fr   hello-world:latest   manager   Shutdown        Complete 15 seconds ago
d5puswsuv948    \_ hello_hello.vktrun1l2rvmov829r3pkq9fr   hello-world:latest   manager   Shutdown        Complete 22 seconds ago
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
```

Figure 2.8. En mode `global`, il y a une copie par nœud, sans avoir à donner de nombre de réplicas.

TD 3 : Traefik et Portainer

J'ai placé le reverse proxy Traefik devant mon cluster Swarm pour joindre chaque application par un nom de domaine, puis j'ai installé Portainer pour gérer le cluster depuis le navigateur.

Exercice 1 : Déployer Traefik

1.1 Relancer le cluster

J'ai relancé le cluster du TD 2 (un manager et 3 nœuds) en publiant le port 80 du manager pour Traefik.

mfp/swarm/compose.yml

```yaml
manager:
  # ...
  ports:
    - "80:80" # Traefik, reachable from the host browser
```

Depuis mfp/swarm

```
docker compose up -d
docker compose exec manager docker node ls
```

Seul le manager a été recréé, et `node ls` montrait bien les 4 nœuds `Ready`.

1.2 Déployer Traefik sur le manager

mfp/swarm/traefik.compose.yml

```yaml
whoami:
  # ...
  deploy:
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.whoami.rule=Host(`whoami.swarm.localhost`)"
      - "traefik.http.routers.whoami.entrypoints=web"
      - "traefik.http.services.whoami.loadbalancer.server.port=80"
```

Comme la stack déclare le réseau `web` en `external`, j'ai dû le créer avant de la déployer, en overlay pour qu'il couvre tous les nœuds. J'ai aussi supprimé ce qui restait du TD 2.

Commandes

```
docker compose exec manager docker network create --driver overlay web
docker compose exec manager docker network ls --filter name=web
docker compose exec manager docker stack rm hello
docker compose exec manager docker service rm web
docker compose cp traefik.compose.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/traefik.compose.yml
↪  traefik
docker compose exec manager docker service ls
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose cp traefik.compose.yml manager:/home/manager/
 swarm-manager-1 Copying traefik.compose.yml to swarm-manager-1:/home/manager/
 swarm-manager-1 Copied traefik.compose.yml to swarm-manager-1:/home/manager/
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager docker stack deploy -c /home/manage
r/traefik.compose.yml traefik
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating service traefik_traefik
Creating service traefik_whoami
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$ docker compose exec manager docker service ls
ID             NAME              MODE         REPLICAS   IMAGE                 PORTS
34nw81r3tlo8   traefik_traefik   replicated   1/1        traefik:v3.6          *:80->80/tcp
mbp8wvpskz9n   traefik_whoami    replicated   1/1        traefik/whoami:latest
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/swarm$
```

Figure 3.1. Seul Traefik publie un port ; whoami reste caché derrière lui.

1.3 Accéder à Traefik depuis le navigateur

Le port 80 était déjà publié depuis la question 1.1. Il restait à déclarer les deux noms dans le fichier hosts de Windows, ouvert en administrateur, parce qu'aucun DNS ne les connaît.

```
C:\Windows\System32\drivers\etc\hosts

127.0.0.1   traefik.swarm.localhost
127.0.0.1   whoami.swarm.localhost
```

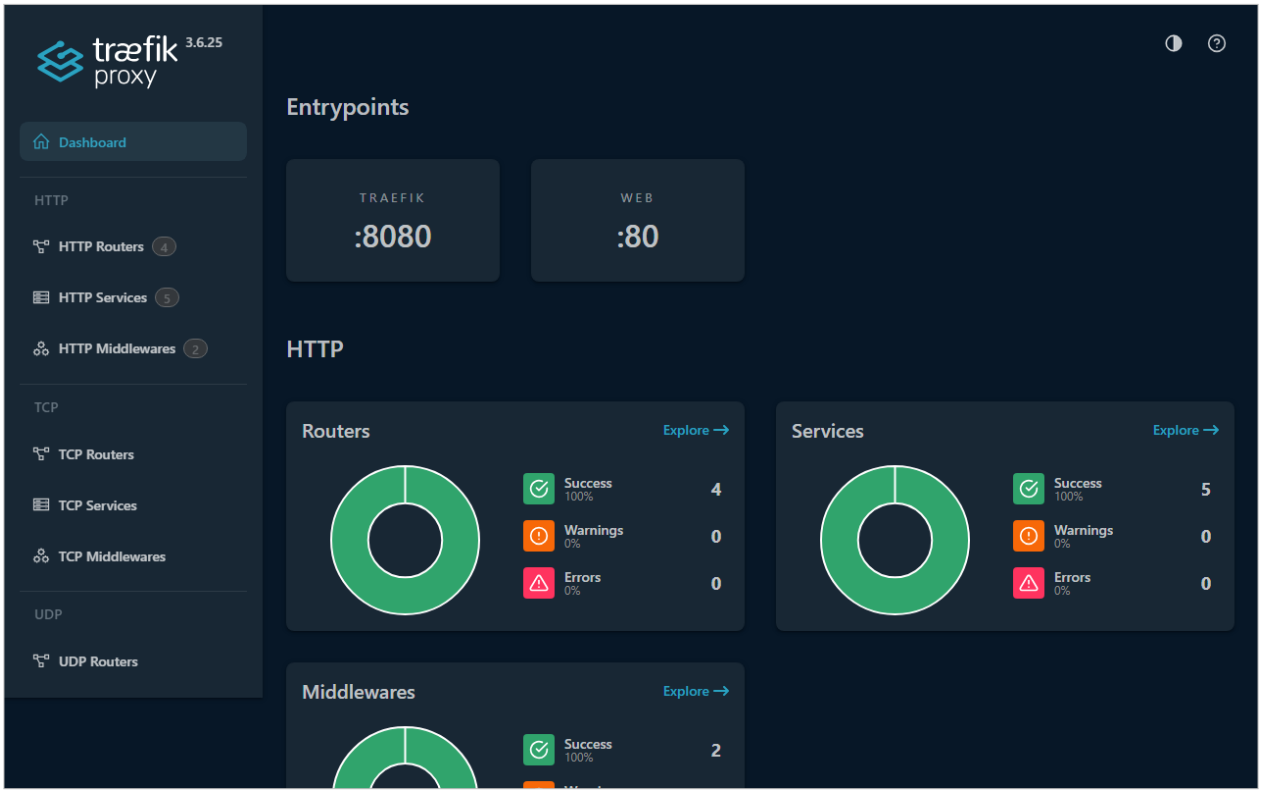


Figure 3.2. Le tableau de bord de Traefik, qui a trouvé les routeurs tout seul grâce aux labels.

```
Hostname: e96fb8cd89f8
IP: 127.0.0.1
IP: ::1
IP: 10.0.2.6
IP: 172.18.0.3
RemoteAddr: 10.0.2.3:34768
GET / HTTP/1.1
Host: whoami.swarm.localhost
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
HeadlessChrome/156.0.0.0 Safari/537.36 Edg/156.0.0.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/
signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: fr,fr-FR;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Sec-Ch-Ua: "Not:A-Brand";v="8", "Chromium";v="156", "Microsoft Edge";v="156"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
Upgrade-Insecure-Requests: 1
X-Forwarded-For: 10.0.0.2
X-Forwarded-Host: whoami.swarm.localhost
X-Forwarded-Port: 80
X-Forwarded-Proto: http
X-Forwarded-Server: 62ea62114d18
X-Real-Ip: 10.0.0.2
```

Figure 3.3. La réponse de whoami. Les en-têtes `X-Forwarded-*` montrent que la requête est bien passée par Traefik.

Exercice 2 : Déployer une autre application

2.1 Tester l'application avec Docker Compose

Depuis voting-app

```
docker compose up -d
docker login
docker compose up -d
docker compose ps --format 'table {{.Name}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
curl -X POST -d vote=a http://localhost:8080/   # Cats, 3 fois
curl -X POST -d vote=b http://localhost:8080/   # Dogs, 2 fois
```

Problème rencontré : Le premier `up -d` a échoué avec `429 Too Many Requests`, car Docker Hub limite les téléchargements anonymes.

Solution : Je me suis connecté avec `docker login`, puis j'ai relancé.

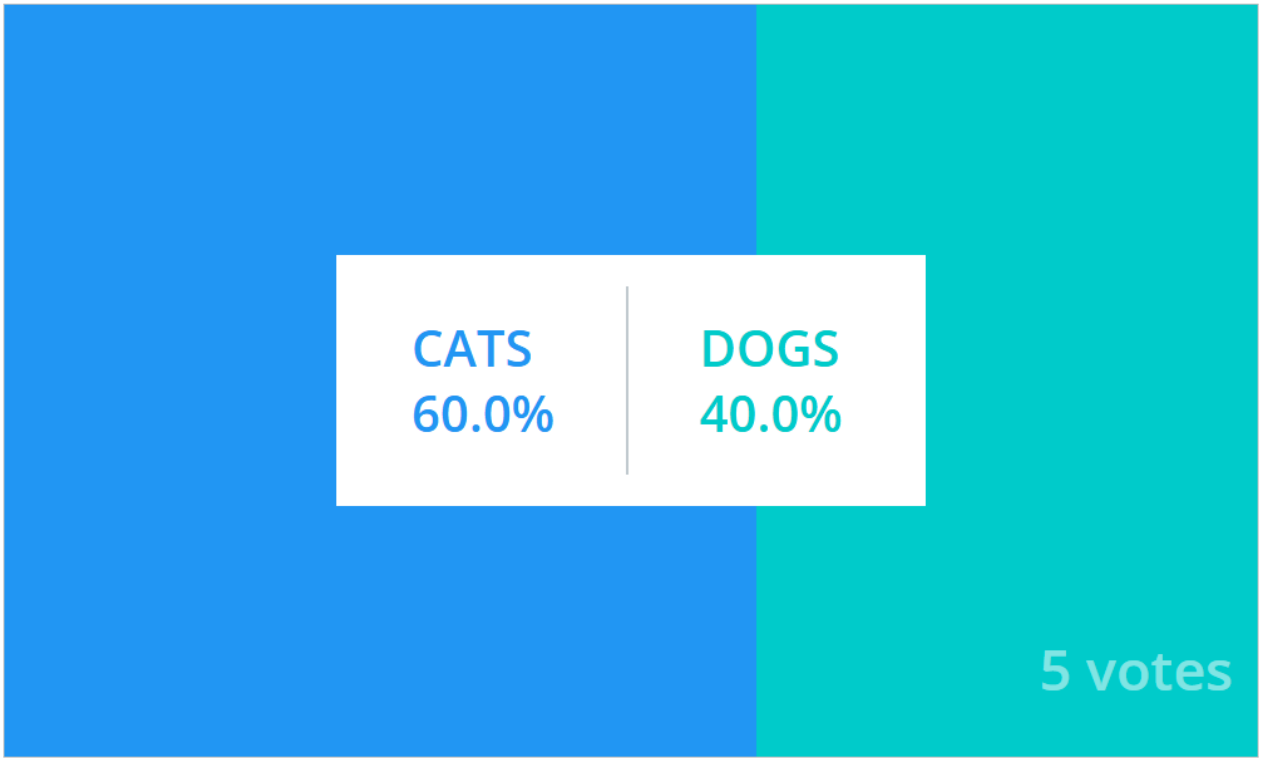


Figure 3.4. Les 5 votes envoyés en local arrivent bien sur la page des résultats.

Commandes

```
docker compose down
```

2.2 Déployer l'application sur le cluster Swarm

L'application a 5 parties. Le site `vote` dépose chaque vote dans `redis`, le `worker` les récupère pour les enregistrer dans la base `db`, et le site `result` lit cette base pour afficher les scores.

Swarm ne sait pas construire d'images, donc la stack fournie utilise des images déjà publiées. Je me suis connecté à Docker Hub dans le manager et j'ai ajouté `--with-registry-auth` pour que les autres nœuds puissent aussi les télécharger.

Depuis mfp/swarm

```
docker compose exec manager docker login
docker compose cp ../../voting-app/docker-stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy --with-registry-auth -c
↪  /home/manager/docker-stack.yml voting
docker compose exec manager docker stack services voting
```

Au bout d'une minute environ, les 5 services tournaient, mais rien ne reliait encore les ports 8080 et 8081 à mon PC.

2.3 Publier le vote et les résultats avec Traefik

Comme pour `whoami`, j'ai remplacé les ports de `vote` et `result` par des labels Traefik et je les ai branchés sur le réseau `web`. Ci-dessous la partie de `vote` ; celle de `result` ne change que par le nom.

voting-app/docker-stack.traefik.yml

```yaml
networks:
  - frontend
  - web
deploy:
  replicas: 2
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.vote.rule=Host(`vote.swarm.localhost`)"
    - "traefik.http.routers.vote.entrypoints=web"
    - "traefik.http.services.vote.loadbalancer.server.port=80"
```

J'ai gardé le nom de stack `voting`, donc Swarm a mis à jour les services au lieu d'en créer d'autres.

Commandes

```
docker compose cp ../../voting-app/docker-stack.traefik.yml manager:/home/manager/
docker compose exec manager docker stack deploy --with-registry-auth -c
↪  /home/manager/docker-stack.traefik.yml voting
docker compose exec manager docker stack services voting
```

hosts

```
127.0.0.1   vote.swarm.localhost
127.0.0.1   result.swarm.localhost
```

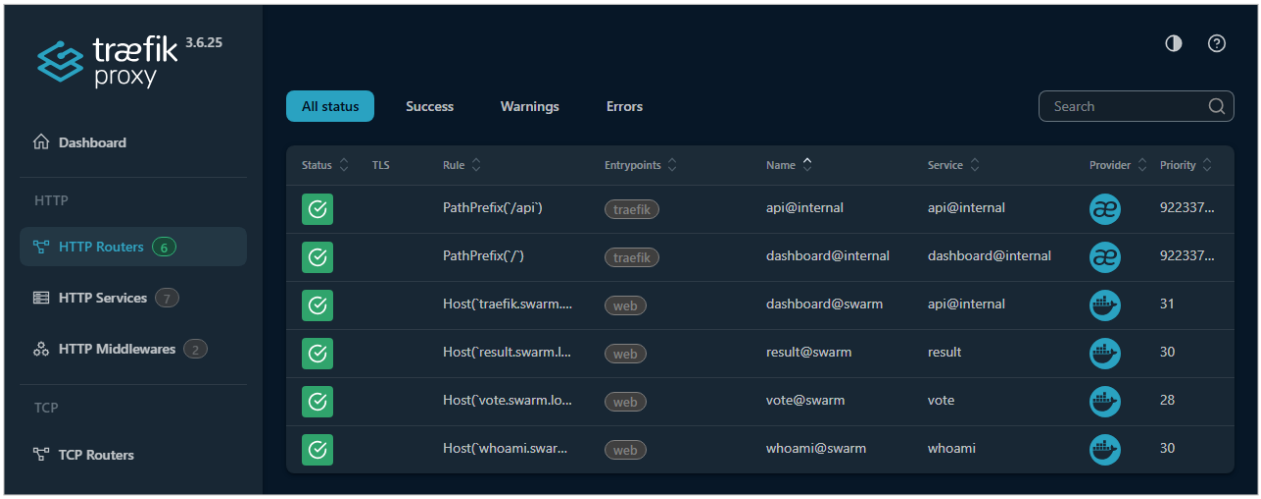


Figure 3.5. Les routeurs `vote@swarm` et `result@swarm` apparaissent dans Traefik.

Commandes

```
curl -X POST -d vote=a http://vote.swarm.localhost/   # Cats, 2 fois
curl -X POST -d vote=b http://vote.swarm.localhost/   # Dogs, 3 fois
```

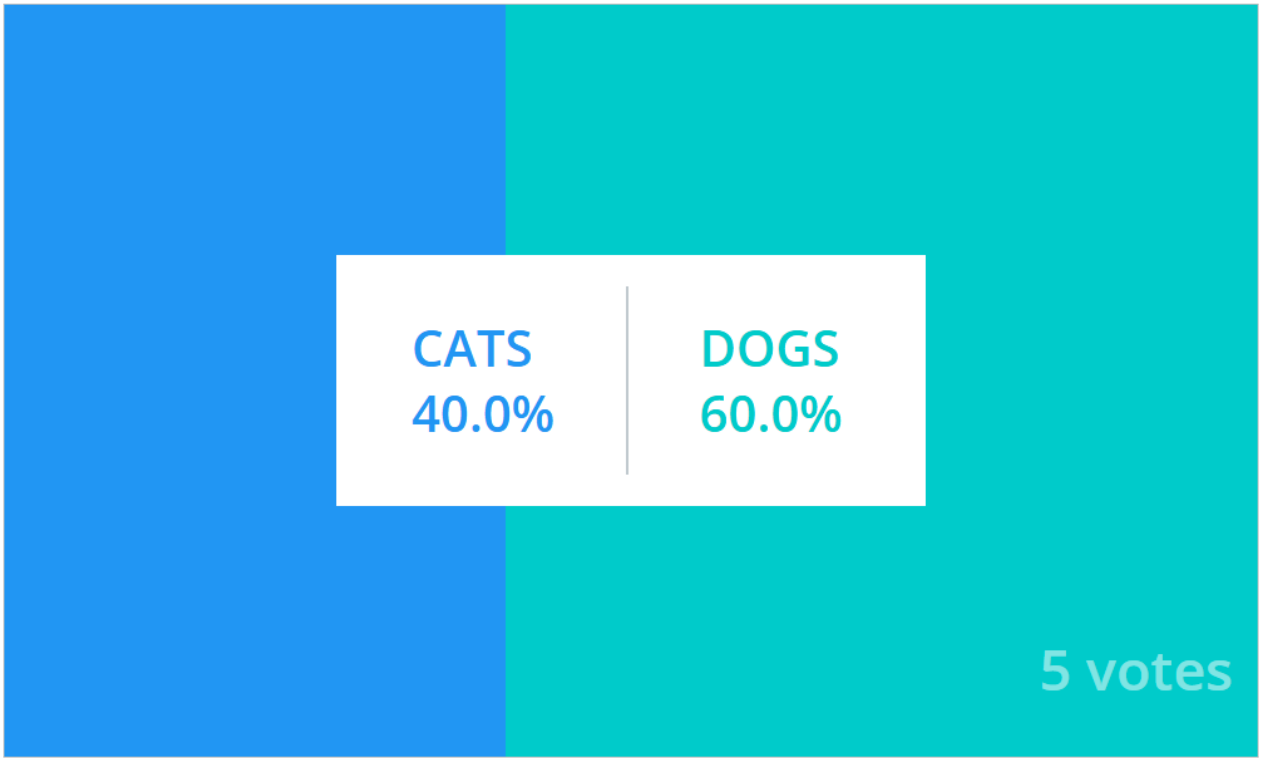


Figure 3.6. Cette fois les résultats sont inversés, parce que la base du cluster ne contient que les votes envoyés au cluster.

Exercice 3 : Déployer Portainer

3.1 Déployer Portainer sur le cluster

J'ai suivi la documentation de Portainer pour Swarm, qui fournit une stack toute prête.

Depuis mfp/swarm

```
curl -sL https://downloads.portainer.io/ce-lts/portainer-agent-stack.yml -o
↪  portainer-agent-stack.yml
docker compose cp portainer-agent-stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy --with-registry-auth -c
↪  /home/manager/portainer-agent-stack.yml portainer
docker compose exec manager docker stack services portainer
docker compose exec manager docker ps --filter name=portainer --format 'table
↪  {{.Names}}\t{{.Image}}\t{{.Status}}'
```

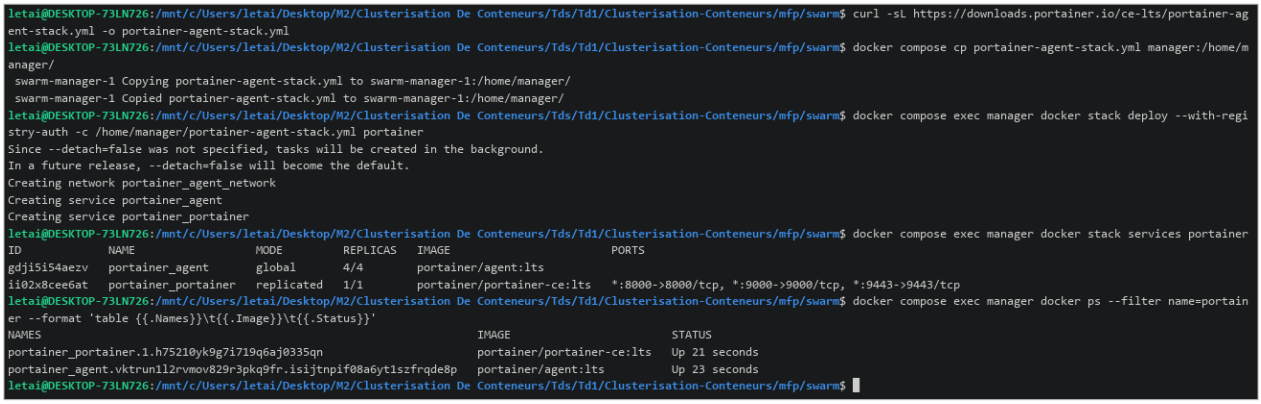


Figure 3.7. Un agent tourne sur chacun des 4 nœuds (mode `global`) et l'interface sur le manager.

3.2 Publier Portainer avec Traefik

Portainer répond sur le port 9000, c'est donc ce port que j'ai mis dans ses labels. Je l'ai aussi ajouté au réseau `web`, sinon Traefik ne pourrait pas le joindre.

mfp/swarm/portainer-agent-stack.traefik.yml

```yaml
networks:
  - agent_network
  - web
deploy:
  # ...
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.portainer.rule=Host(`portainer.swarm.localhost`)"
    - "traefik.http.routers.portainer.entrypoints=web"
    - "traefik.http.services.portainer.loadbalancer.server.port=9000"
```

Commandes

```
docker compose cp portainer-agent-stack.traefik.yml manager:/home/manager/
docker compose exec manager docker stack deploy --with-registry-auth -c
↪  /home/manager/portainer-agent-stack.traefik.yml portainer
docker compose exec manager docker stack services portainer
```

3.3 Ouvrir Portainer dans le navigateur

hosts

```
127.0.0.1   portainer.swarm.localhost
```

Au premier lancement, Portainer demande un setup token, que j'ai récupéré dans ses logs.

Commandes

```
docker compose exec manager docker service logs --raw portainer_portainer 2>&1 | grep -A2
↪  setup_token=
```

J'ai dû créer le compte `admin` dans les 5 minutes qui suivent le démarrage, sinon Portainer s'arrête par sécurité.

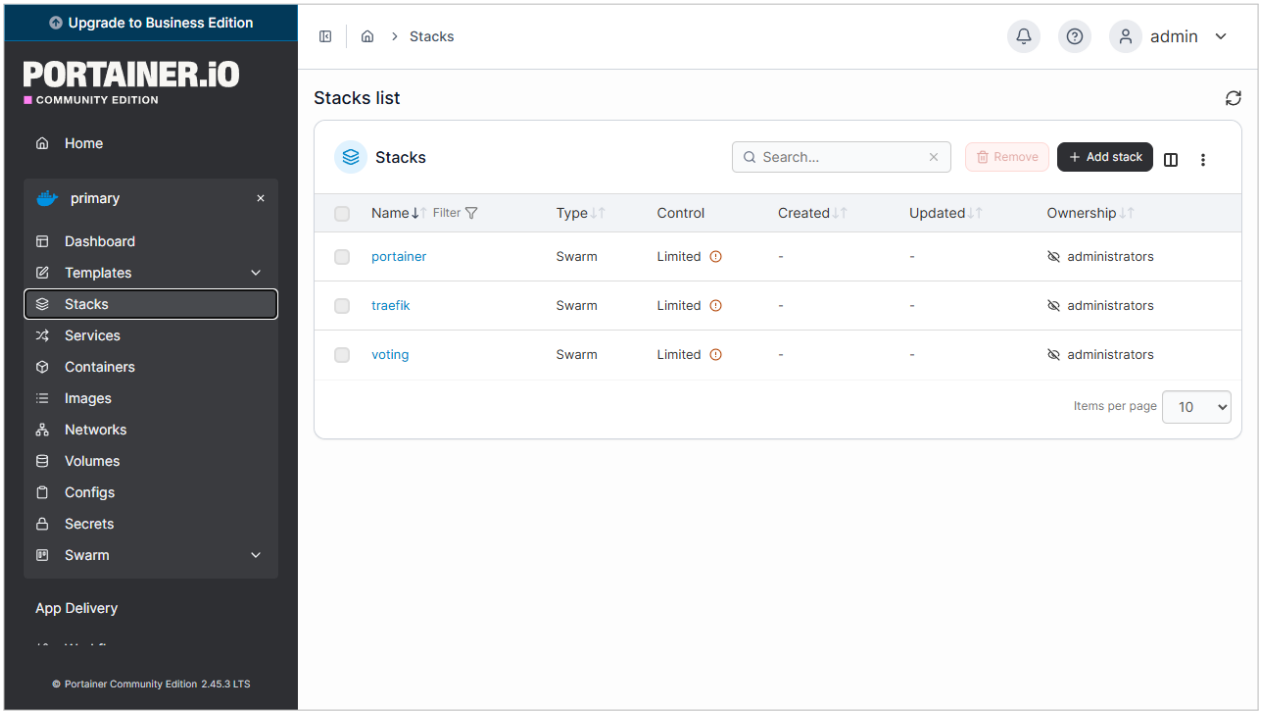


Figure 3.8. Je retrouve mes 3 stacks, marquées « Limited » parce qu'elles ont été créées en ligne de commande.

Exercice 4 : Redéployer depuis Portainer

4.1 Supprimer la stack depuis le manager

Depuis mfp/swarm

```
docker compose exec manager docker stack ls
docker compose exec manager docker stack rm voting
docker compose exec manager docker stack ls
```

4.2 Republier la stack depuis Portainer

J'ai envoyé le fichier `docker-stack.traefik.yml` de la question 2.3, pour garder les adresses du vote et des résultats.

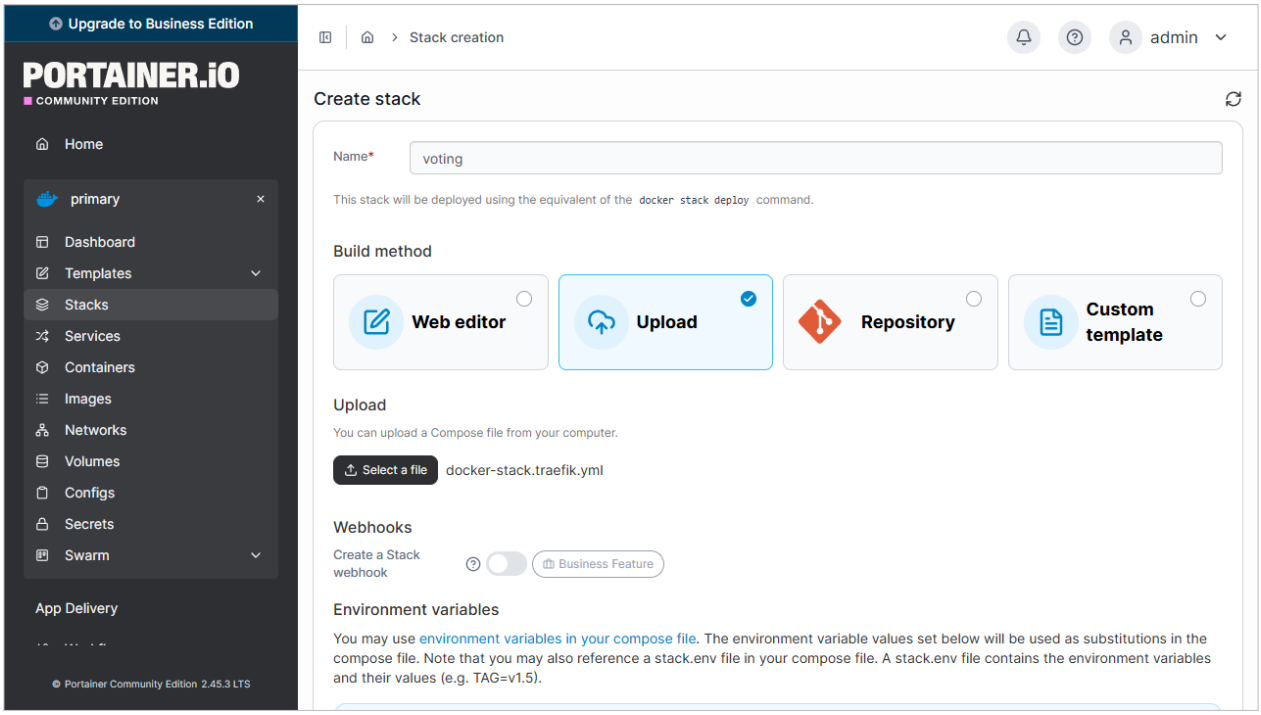


Figure 3.9. La création de la stack `voting` dans Portainer, par envoi du fichier.

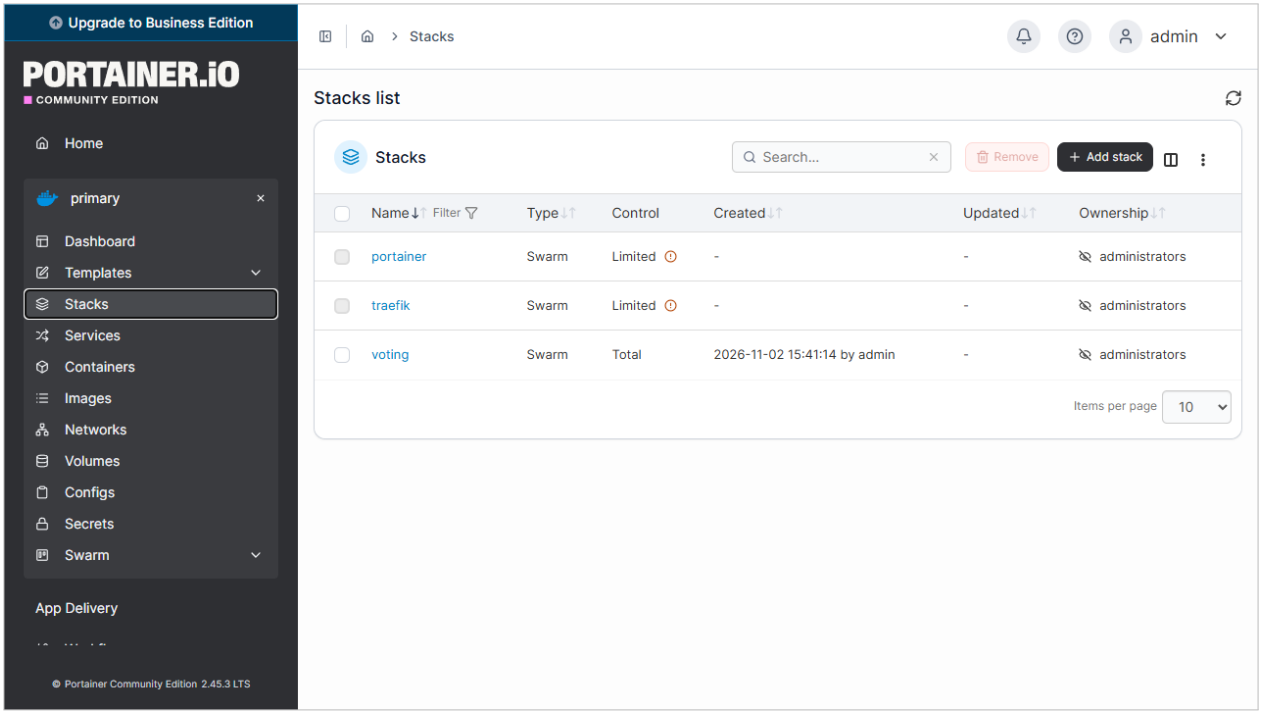


Figure 3.10. Cette fois `voting` est en contrôle « Total », car c'est Portainer qui l'a créée.

4.3 Vérifier l'accès depuis le navigateur

Le vote et les résultats répondaient de nouveau sur leurs adresses.

Problème rencontré : La page des résultats affichait « No votes yet », car la base avait été recréée sur `node2` avec un volume neuf.

Solution : Rien à corriger, les anciens votes sont restés dans le volume de `node3`.

4.4 Déployer l'API de MFP depuis Portainer

4.4.a Création de la stack

Comme pour l'application de vote, le serveur utilise une image déjà publiée, celle que j'avais mise sur GHCR. Il passe par Traefik au lieu de publier le port 3000.

mfp/swarm/mfp-api.stack.yml

```yaml
server:
  # ...
  deploy:
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.mfp-api.rule=Host(`api.swarm.localhost`)"
      - "traefik.http.routers.mfp-api.entrypoints=web"
      - "traefik.http.services.mfp-api.loadbalancer.server.port=3000"
```

J'ai déployé la stack `mfp` depuis Portainer, de la même façon qu'à la question 4.2.

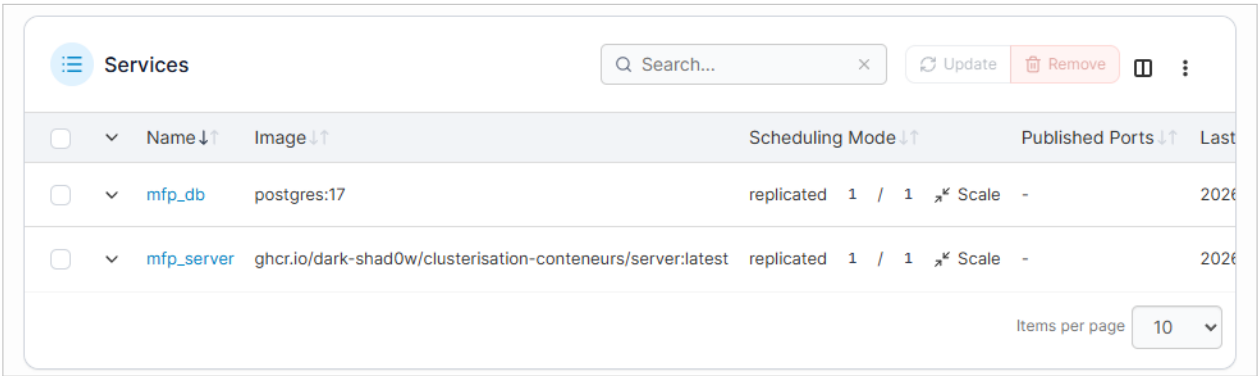


Figure 3.11. La base et le serveur de MFP tournent, sans aucun port publié.

Problème rencontré : Au premier démarrage, le serveur s'arrêtait sur `getaddrinfo ENOTFOUND db`, parce que la base n'était pas encore prête.

Solution : Je n'ai rien eu à faire, car Swarm a relancé le serveur, qui s'est connecté au deuxième essai.

4.4.b Test avec Bruno

hosts

```
127.0.0.1   api.swarm.localhost
```

Pour ne pas abîmer ma collection Bruno, j'en ai fait une copie et j'ai changé son adresse avec `sed`.

Depuis mfp

```
cp -r mfp-bruno-api-test-collection ~/mfp-bruno-swarm
sed -i 's#http://localhost:3000#http://api.swarm.localhost#' ~/mfp-bruno-swarm/*.yml
cd ~/mfp-bruno-swarm && bru run
```

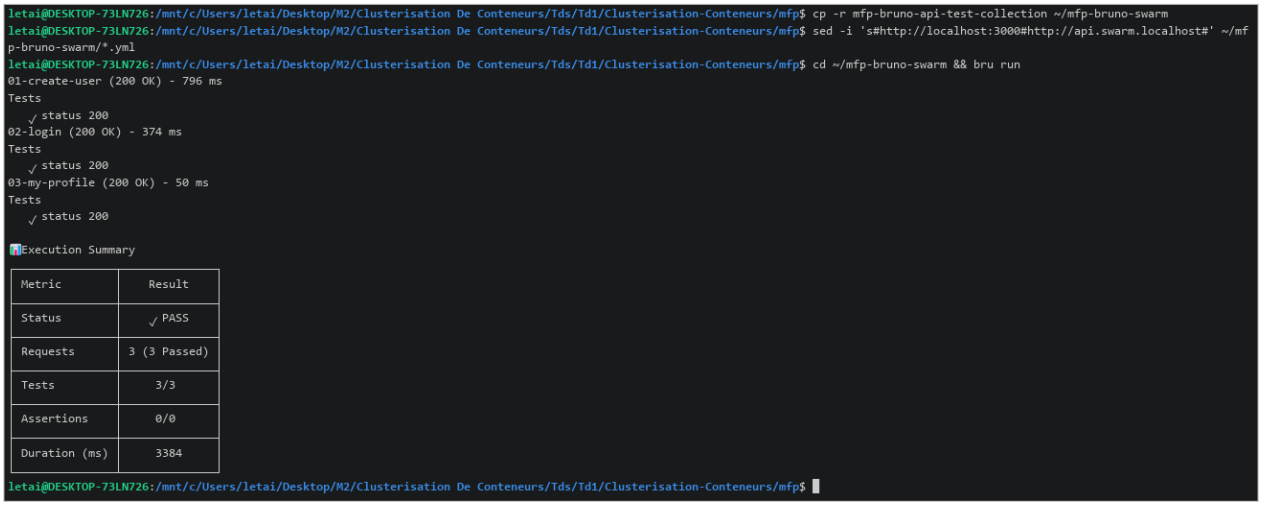


Figure 3.12. Les 3 requêtes passent, donc l'API du cluster répond bien depuis mon PC.

TD 4 : Secrets et configs Docker Swarm

Swarm peut stocker lui-même deux sortes de données pour les conteneurs. Les secrets servent aux informations sensibles et ne peuvent plus être relus une fois créés ; les configs se relisent mais ne se modifient pas. Dans les deux cas, ils arrivent sous forme de fichiers dans les conteneurs, quel que soit le nœud.

Prérequis : relancer le cluster et passer en HTTPS

0.1 Relancer le cluster

swarm/compose.yml

```yaml
manager:
  # ...
  ports:
    - "80:80" # Traefik, reachable from the host browser
    - "443:443" # Traefik HTTPS (TD4: Authelia only works over https)
```

Commandes

```
docker compose up -d
docker compose ps
docker compose exec manager docker node ls
```

Problème rencontré : Le manager a été recréé pour le port 443 et les nœuds `node1` à `node3` sont passés en `Down`, parce que les adresses IP avaient changé.

Solution : J'ai fait quitter le swarm à tous les nœuds et j'ai refait le cluster.

Commandes

```
docker compose exec node1 docker swarm leave --force
docker compose exec node2 docker swarm leave --force
docker compose exec node3 docker swarm leave --force
docker compose exec manager docker swarm leave --force

docker compose exec manager docker swarm init
TOKEN=$(docker compose exec manager docker swarm join-token -q worker)
docker compose exec node1 docker swarm join --token $TOKEN manager:2377
docker compose exec node2 docker swarm join --token $TOKEN manager:2377
docker compose exec node3 docker swarm join --token $TOKEN manager:2377

docker compose exec manager docker node ls
```

En quittant l'ancien cluster, j'ai perdu le réseau `web` et toutes les stacks, mais les volumes sont restés sur les nœuds.

0.2 Créer le certificat

Authelia refuse de démarrer sans adresse en `https://`. J'ai donc fabriqué une CA locale, reconnue par Windows, et un certificat `*.swarm.localhost` signé par elle.

Commandes

```
mkdir -p certs
openssl req -x509 -newkey rsa:2048 -noenc -days 3650 -subj "/CN=Swarm Local CA" -keyout
↪  certs/ca.key -out certs/ca.crt
openssl req -newkey rsa:2048 -noenc -subj "/CN=*.swarm.localhost" -keyout
↪  certs/swarm.localhost.key -out certs/swarm.localhost.csr
openssl x509 -req -in certs/swarm.localhost.csr -CA certs/ca.crt -CAkey certs/ca.key
↪  -CAcreateserial -days 365 -extfile <(printf
↪  "subjectAltName=DNS:*.swarm.localhost,DNS:swarm.localhost") -out
↪  certs/swarm.localhost.crt
ls certs
openssl verify -CAfile certs/ca.crt certs/swarm.localhost.crt
certutil.exe -user -addstore Root "$(wslpath -w certs/ca.crt)"
```



Figure 4.1. Le certificat est bien signé par ma CA.

0.3 Le certificat dans le cluster : un secret et des configs

Comme la clé privée est sensible, je l'ai mise dans un secret. Le certificat est public, donc il va dans une config, tout comme `traefik-tls.yml` qui indique à Traefik quel certificat utiliser.

Commandes

```
docker compose exec manager docker network create --driver overlay web
docker compose exec -T manager docker config create swarm_localhost_crt - <
↪  certs/swarm.localhost.crt
docker compose exec -T manager docker config create traefik_tls - < traefik-tls.yml
docker compose exec -T manager docker secret create swarm_localhost_key - <
↪  certs/swarm.localhost.key
docker compose exec manager docker config ls
docker compose exec manager docker secret ls
docker compose exec manager docker config inspect --pretty traefik_tls
docker compose exec manager docker secret inspect --pretty swarm_localhost_key
```

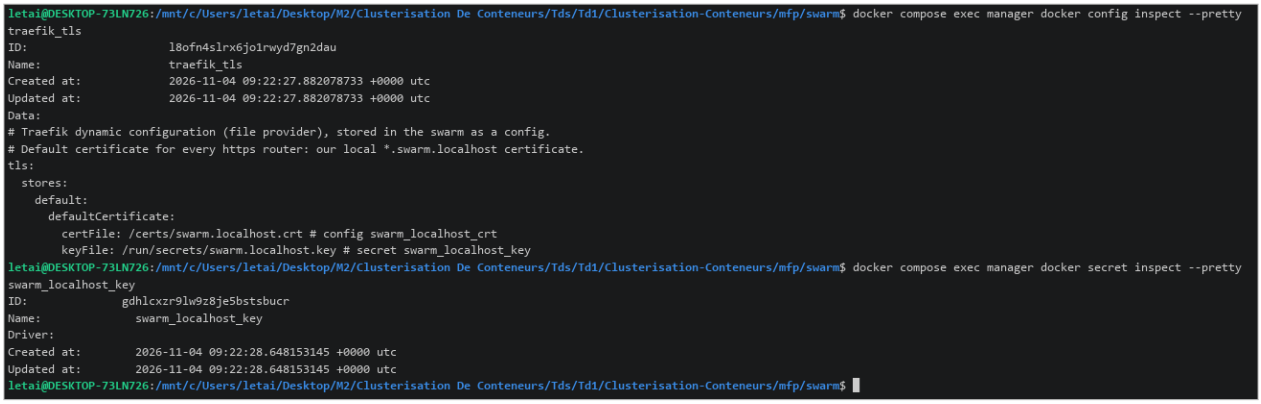


Figure 4.2. La config montre son contenu, le secret seulement son nom et ses dates.

0.4 Traefik en HTTPS

swarm/traefik-https.stack.yml

```yaml
networks:
  web:
    aliases:
      - authelia.swarm.localhost
configs:
  - source: traefik_tls
    target: /etc/traefik/tls.yml
  - source: swarm_localhost_crt
    target: /certs/swarm.localhost.crt
secrets:
  - source: swarm_localhost_key
    target: swarm.localhost.key # mounted in /run/secrets/
command:
  - "--entrypoints.websecure.address=:443"
  - "--entrypoints.websecure.http.tls=true"
```

Commandes

```
docker compose cp traefik-https.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/traefik-https.stack.yml
↪  traefik
docker compose exec manager docker service ls
curl -s --cacert certs/ca.crt https://whoami.swarm.localhost | head -4
curl -sI http://dozzle.swarm.localhost | head -2
```

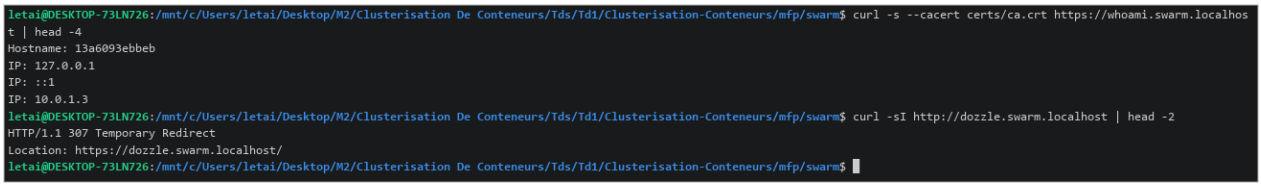


Figure 4.3. `whoami` répond en HTTPS, et une adresse en http est redirigée vers https.

0.5 Relancer Portainer et ajouter les adresses

Commandes

```
docker compose exec manager docker volume ls
docker compose cp portainer-agent-stack.traefik.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c
↪  /home/manager/portainer-agent-stack.traefik.yml portainer
docker compose exec manager docker service ls
```

Portainer a retrouvé son compte `admin`, resté dans son volume sur le manager.

hosts (Windows)

```
127.0.0.1 dozzle.swarm.localhost
127.0.0.1 authelia.swarm.localhost
127.0.0.1 outline.swarm.localhost
```

Exercice 1 : Déployer des logiciels

1.1 Dozzle avec un utilisateur simple

1.1.a Le fichier users.yml

Comme l'indique sa doc, Dozzle génère lui-même le fichier `users.yml`.

Commandes

```
docker run --rm amir20/dozzle generate admin --password dozzle-admin-2026 --name "Admin"
```

1.1.b Le secret, créé dans Portainer

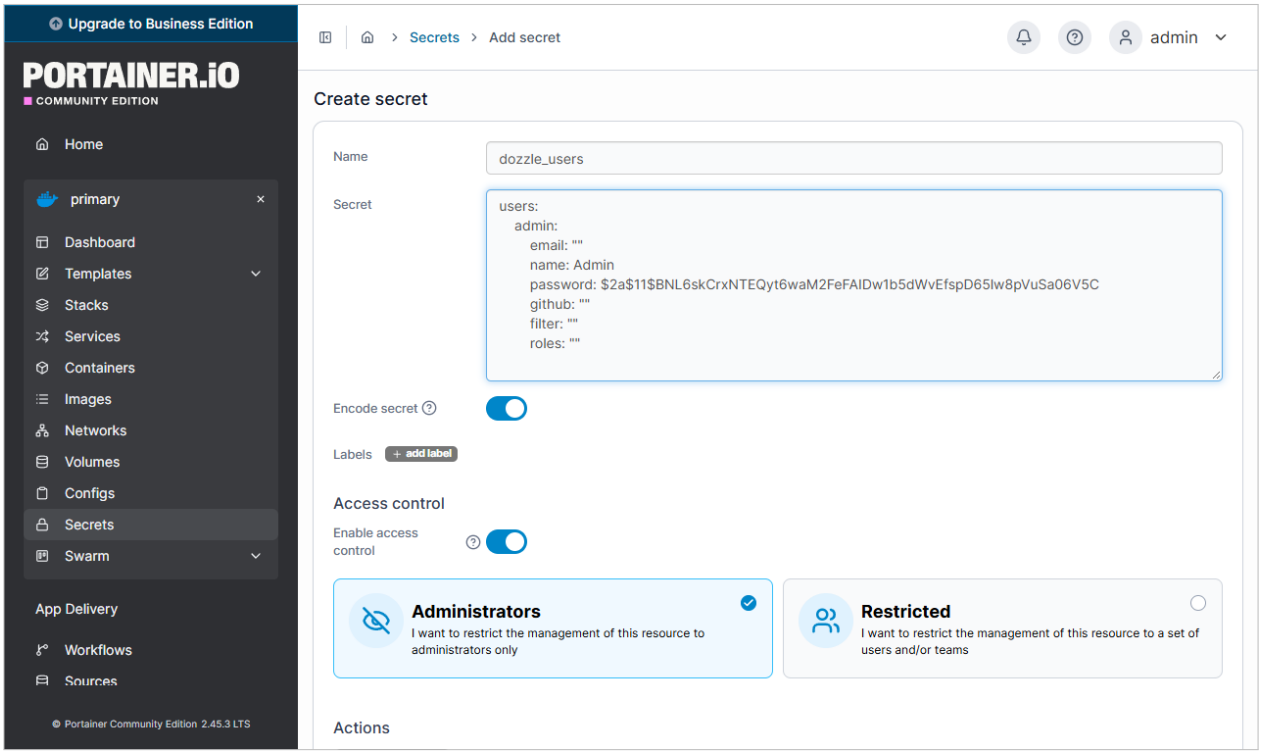


Figure 4.4. Le secret `dozzle_users` créé dans Portainer, avec le contenu de `users.yml`.

1.1.c La stack

swarm/dozzle.stack.yml

```yaml
environment:
  DOZZLE_MODE: swarm # one Dozzle per node, they find each other on the "dozzle" network
  DOZZLE_AUTH_PROVIDER: simple # login form, users read from /data/users.yml
# ...
secrets:
  - source: dozzle_users
    target: /data/users.yml # where Dozzle looks for its users
```

Commandes

```
docker compose cp dozzle.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/dozzle.stack.yml dozzle
docker compose exec manager docker service ps dozzle_dozzle
```

1.1.d Test dans le navigateur

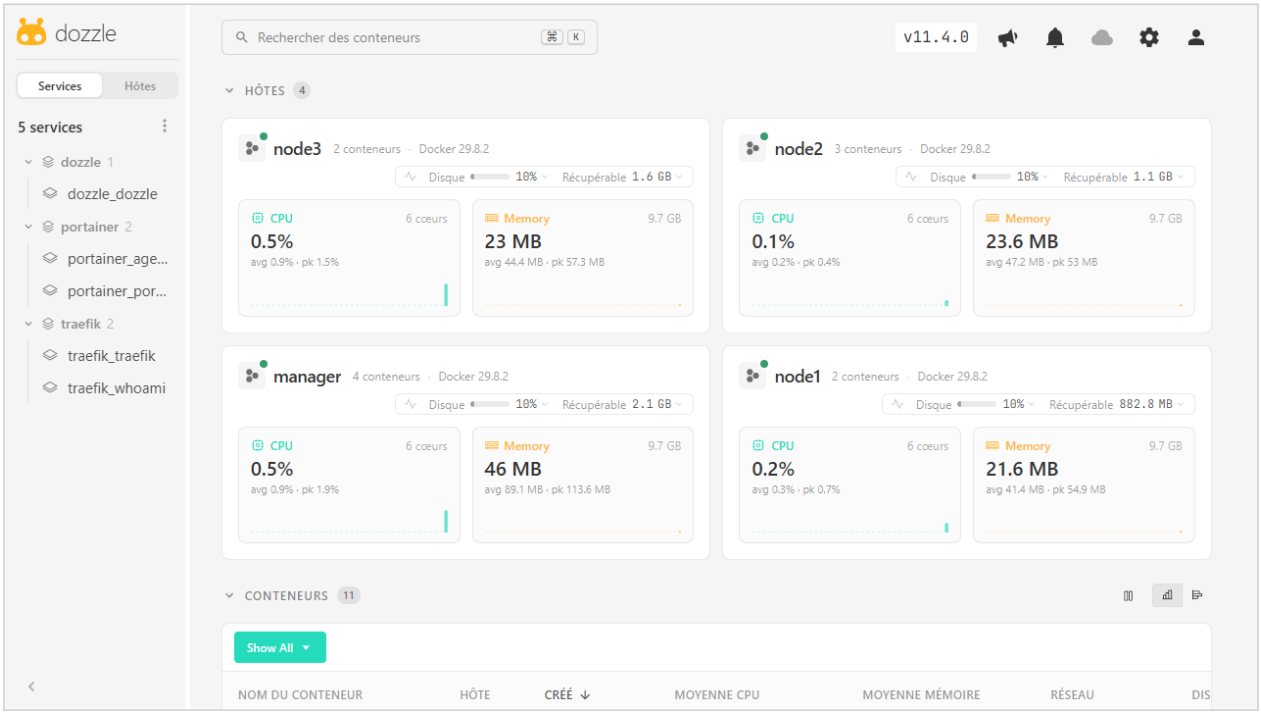


Figure 4.5. Dozzle fonctionne sur `dozzle.swarm.localhost` avec le compte `admin` et voit les 4 nœuds.

1.2 Authelia et connexion OpenID Connect

Avec OpenID Connect, Dozzle ne gère plus de comptes. Il renvoie vers Authelia, qui vérifie le mot de passe et lui rend un jeton signé.

1.2.a Les clés d'Authelia

J'ai envoyé les clés directement d'`openssl` dans Swarm, pour qu'elles ne soient jamais écrites sur le disque.

Commandes

```
for s in authelia_jwt_secret authelia_session_secret authelia_storage_key
↪  authelia_oidc_hmac; do openssl rand -hex 32 | docker compose exec -T manager docker
↪  secret create $s -; done
openssl genrsa 2048 | docker compose exec -T manager docker secret create
↪  authelia_oidc_key -
docker compose exec -T manager docker config create swarm_ca - < certs/ca.crt
docker compose exec manager docker secret ls
```

1.2.b Le compte dans un secret

Commandes

```
docker run --rm authelia/authelia:4.39 authelia crypto hash generate argon2 --password
↪  'ahmed-authelia-2026'
cat secrets/users_database.yml
docker compose exec -T manager docker secret create authelia_users - <
↪  secrets/users_database.yml
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker run --rm authelia/authelia:4.39 authelia crypto hash
 generate argon2 --password 'ahmed-authelia-2026'
Digest: $argon2id$v=19$m=65536,t=3,p=4$wIksLjZLI6+vY01GqMoWsQ$ahFnNv1pcu5dAPHvB8nwz2uGvGPomDFPdsuiGRwTZhM
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ cat secrets/users_database.yml
users:
  ahmed:
    displayname: 'Ahmed'
    password: '$argon2id$v=19$m=65536,t=3,p=4$wIksLjZLI6+vY01GqMoWsQ$ahFnNv1pcu5dAPHvB8nwz2uGvGPomDFPdsuiGRwTZhM'
    email: 'ahmed@swarm.localhost'
    groups:
      - 'admins'
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec -T manager docker secret create autheli
a_users - < secrets/users_database.yml
tmnskc4ujpcvz6womxqc99wp1
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$
```

Figure 4.6. Le compte `ahmed`, du groupe `admins`, devient le secret `authelia_users`.

1.2.c Le client OIDC de Dozzle

Dozzle doit connaître le client secret en clair, je l'ai donc mis dans un secret. Authelia, lui, n'en garde que l'empreinte dans sa configuration.

Commandes

```
openssl rand -hex 36 | tr -d "\n" > secrets/dozzle_oidc_secret
docker run --rm authelia/authelia:4.39 authelia crypto hash generate pbkdf2 --variant
↪  sha512 --password "$(cat secrets/dozzle_oidc_secret)"
docker compose exec -T manager docker secret create dozzle_oidc_secret - <
↪  secrets/dozzle_oidc_secret
```

1.2.d La configuration dans une config

Cette configuration ne contient rien de secret, je pouvais donc la mettre dans une config.

swarm/authelia.configuration.yml

```
session:
  cookies:
    - domain: 'swarm.localhost'
```

```
      authelia_url: 'https://authelia.swarm.localhost'
# ...
    clients:
      - client_id: 'dozzle'
        client_secret: '$pbkdf2-sha512$310000$zQ.7CBa0mgiu8k8Zm3dTBA$cliYtETUut.EYaAb1C1x5
        ↪  G0d8nJ7TBDq3ewmffU1bvgb/PbES0giHXdRx6Acqbgoa0V8yBz2Wk12VnIo7jdzLg'
        redirect_uris:
          - 'https://dozzle.swarm.localhost/api/auth/callback'
```

1.2.e Le déploiement

swarm/authelia.stack.yml

```
environment:
  AUTHELIA_SESSION_SECRET_FILE: /run/secrets/authelia_session_secret
  AUTHELIA_STORAGE_ENCRYPTION_KEY_FILE: /run/secrets/authelia_storage_key
  # ...
configs:
  - source: authelia_config
    target: /config/configuration.yml
secrets:
  - source: authelia_users
    target: /config/users_database.yml # the account(s)
  - authelia_session_secret
  # ...
```

Commandes

```
docker compose exec -T manager docker config create authelia_config - <
↪  authelia.configuration.yml
docker compose cp authelia.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/authelia.stack.yml
↪  authelia
docker compose exec manager docker service ls --filter name=authelia
docker compose exec manager docker service logs --raw authelia_authelia 2>&1 | tail -4
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec -T manager docker config create authelia_config - < authelia.configuration.yml
6uu2f2yvqgbya4r16uripjh6y
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose cp authelia.stack.yml manager:/home/manager/
 swarm-manager-1 Copying authelia.stack.yml to swarm-manager-1:/home/manager/
 swarm-manager-1 Copied authelia.stack.yml to swarm-manager-1:/home/manager/
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker stack deploy -c /home/manager/authelia.stack.yml authelia
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating service authelia_authelia
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker service ls --filter name=authelia
ID             NAME               MODE         REPLICAS   IMAGE                  PORTS
69dvojw9oja0   authelia_authelia  replicated   1/1        authelia/authelia:4.39
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker service logs --raw authelia_authelia 2>&1 | tail -4
time="2026-11-04T10:38:29+01:00" level=info msg="Storage schema migration from 0 to 29 is being attempted"
time="2026-11-04T10:38:29+01:00" level=info msg="Storage schema migration from 0 to 29 is complete"
time="2026-11-04T10:38:31+01:00" level=info msg="Listening for non-TLS connections on '[::]:9091' path '/'" server=main service=server
time="2026-11-04T10:38:31+01:00" level=info msg="Startup complete"
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$
```

Figure 4.7. Authelia démarre sans erreur avec sa config et ses secrets.

1.2.f Brancher Dozzle sur Authelia

Dozzle appelle lui-même `https://authelia.swarm.localhost`, mais ce nom n'existe pas dans les conteneurs. C'est pour ça que j'avais donné cet alias à Traefik sur le réseau `web`, et la config `swarm_ca` permet à Dozzle d'accepter mon certificat.

swarm/dozzle.oidc.stack.yml

```yaml
environment:
  DOZZLE_AUTH_PROVIDER: oidc # Authelia owns the accounts, users.yml is no longer read
  DOZZLE_AUTH_OIDC_ISSUER: https://authelia.swarm.localhost
  DOZZLE_AUTH_OIDC_CLIENT_ID: dozzle
  DOZZLE_AUTH_OIDC_CLIENT_SECRET_FILE: /run/secrets/dozzle_oidc_secret
  SSL_CERT_FILE: /certs/ca.crt
# ...
configs:
  - source: swarm_ca
    target: /certs/ca.crt
```

Commandes

```
docker compose cp dozzle.oidc.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/dozzle.oidc.stack.yml
↪  dozzle
docker compose exec manager docker service ps dozzle_dozzle --filter desired-state=running
docker compose exec manager docker service logs --raw --since 2m dozzle_dozzle 2>&1 | grep
↪  -iE 'oidc|auth|error|fatal' | head -8
```

```
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose cp dozzle.oidc.stack.yml manager:/home/manag
er/
 swarm-manager-1 Copying dozzle.oidc.stack.yml to swarm-manager-1:/home/manager/
 swarm-manager-1 Copied dozzle.oidc.stack.yml to swarm-manager-1:/home/manager/
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker stack deploy -c /home/ma
nager/dozzle.oidc.stack.yml dozzle
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Updating service dozzle_dozzle (id: fsw1thgvkzmjnuwpcn7aqlv4b)
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker service ps dozzle_dozzle
 --filter desired-state=running
ID             NAME                                    IMAGE                NODE      DESIRED STATE   CURRENT STATE            ERROR     PORTS
3ajebiwrfo45   dozzle_dozzle.spjvxolb649yacz4jcv685jk3   amir20/dozzle:latest   node1     Running         Running 36 seconds ago
omrjaohv2wz2   dozzle_dozzle.w7ufj1jhxaf70ep1re4ouyllh   amir20/dozzle:latest   manager   Running         Running 42 seconds ago
bvdx7pdznxuk   dozzle_dozzle.wkc8d6w02b5j4ksw9jeh7l3bm   amir20/dozzle:latest   node2     Running         Running 30 seconds ago
m0qq5c5jmcta   dozzle_dozzle.yc475joofqmkybq45qayead8f   amir20/dozzle:latest   node3     Running         Running 20 seconds ago
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$ docker compose exec manager docker service logs --raw --sin
ce 2m dozzle_dozzle 2>&1 | grep -iE 'oidc|auth|error|fatal' | head -8
{"level":"warn","version":"v11.4.0","file":"users.yml","time":"2026-11-04T09:40:05Z","message":"Ignoring the user database: with --auth-provider oidc every user comes from the token"}
{"level":"info","version":"v11.4.0","issuer":"https://authelia.swarm.localhost","rolesClaim":"dozzle_roles, resource_access.dozzle.roles, roles","time":"2026-11-04T09:40:05Z","message":
"Using OpenID Connect authentication; users and roles come from the token"}
{"level":"warn","version":"v11.4.0","file":"users.yml","time":"2026-11-04T09:39:55Z","message":"Ignoring the user database: with --auth-provider oidc every user comes from the token"}
{"level":"info","version":"v11.4.0","issuer":"https://authelia.swarm.localhost","rolesClaim":"dozzle_roles, resource_access.dozzle.roles, roles","time":"2026-11-04T09:39:55Z","message":
"Using OpenID Connect authentication; users and roles come from the token"}
{"level":"warn","version":"v11.4.0","file":"users.yml","time":"2026-11-04T09:39:49Z","message":"Ignoring the user database: with --auth-provider oidc every user comes from the token"}
{"level":"info","version":"v11.4.0","issuer":"https://authelia.swarm.localhost","rolesClaim":"dozzle_roles, resource_access.dozzle.roles, roles","time":"2026-11-04T09:39:49Z","message":
"Using OpenID Connect authentication; users and roles come from the token"}
{"level":"warn","version":"v11.4.0","file":"users.yml","time":"2026-11-04T09:39:43Z","message":"Ignoring the user database: with --auth-provider oidc every user comes from the token"}
{"level":"info","version":"v11.4.0","issuer":"https://authelia.swarm.localhost","rolesClaim":"dozzle_roles, resource_access.dozzle.roles, roles","time":"2026-11-04T09:39:43Z","message":
"Using OpenID Connect authentication; users and roles come from the token"}
letai@DESKTOP-73LN726:/mnt/c/Users/letai/Desktop/M2/Clusterisation De Conteneurs/Tds/Td1/Clusterisation-Conteneurs/mfp/swarm$
```

Figure 4.8. Dozzle est bien passé en OpenID Connect, avec Authelia comme émetteur.

1.2.g Rôles manquants et mise à jour de la config

Problème rencontré : Au retour d'Authelia, Dozzle refusait la connexion avec l'erreur `no roles claim found in the ID token`.

Solution : J'ai ajouté dans Authelia un attribut `dozzle_roles` calculé à partir des groupes, et Dozzle le demande avec `DOZZLE_AUTH_OIDC_SCOPES: dozzle_roles`.

Comme une config ne peut pas être modifiée, j'ai dû la supprimer puis la recréer avec ces ajouts.

swarm/authelia.configuration.yml

```yaml
definitions:
  user_attributes:
    dozzle_roles:
      expression: '"admins" in groups ? ["all"] : []'
# ...
    claims_policies:
      dozzle:
        id_token: ['dozzle_roles']
        # ...
```

Problème rencontré : Le premier `config rm` a été refusé, parce que la config était encore utilisée.
Solution : J'ai d'abord retiré la stack `authelia`, sans perdre son volume.

Commandes

```
docker compose exec manager docker config rm authelia_config
docker compose exec manager docker stack rm authelia
docker compose exec manager docker config rm authelia_config
docker compose exec -T manager docker config create authelia_config - <
↪  authelia.configuration.yml
docker compose exec manager docker stack deploy -c /home/manager/authelia.stack.yml
↪  authelia
docker compose exec manager docker service ls --filter name=authelia
docker compose cp dozzle.oidc.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/dozzle.oidc.stack.yml
↪  dozzle
docker compose exec manager docker service ls --filter name=dozzle
```

1.2.h Connexion à Dozzle par Authelia

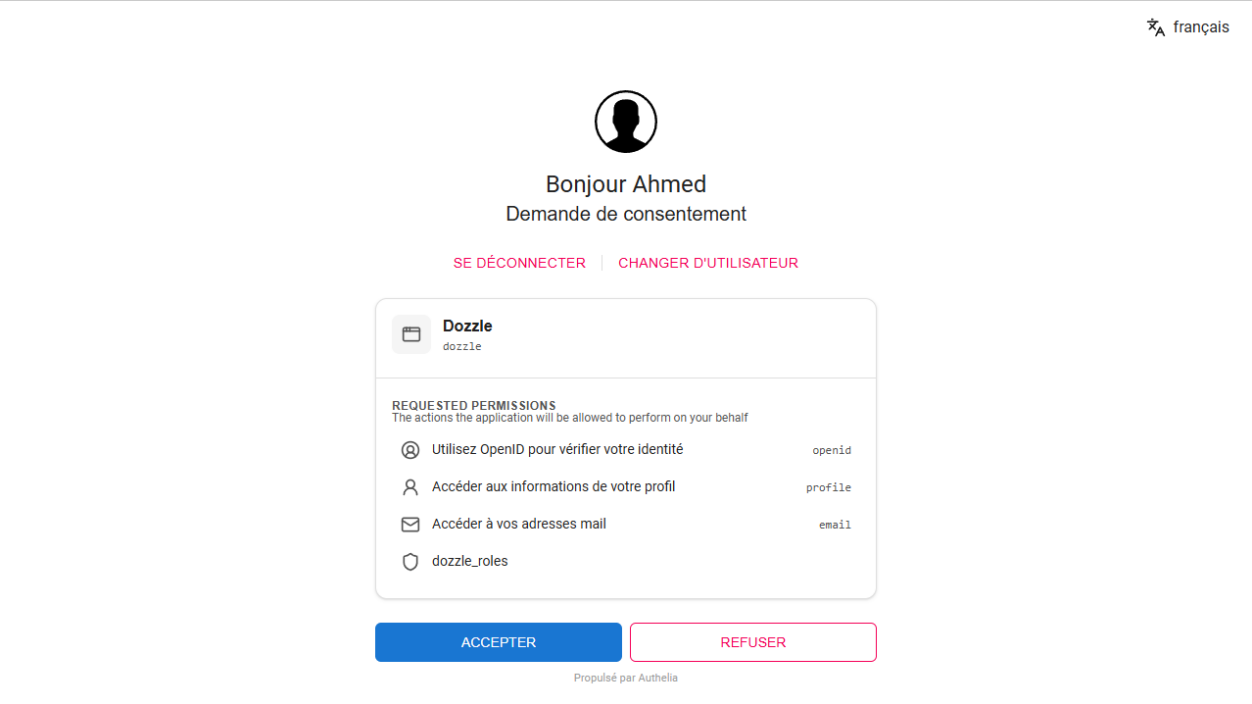


Figure 4.9. Connecté sur `authelia.swarm.localhost` avec le compte `ahmed`.

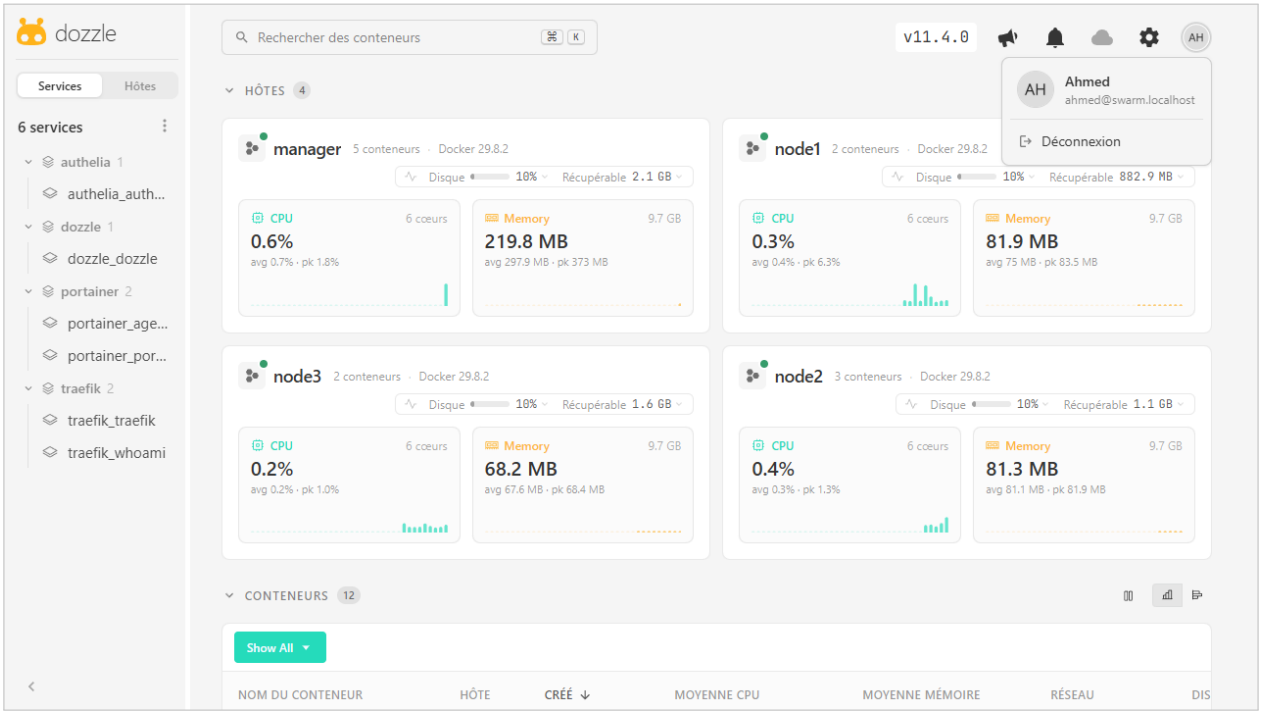


Figure 4.10. Je suis connecté à Dozzle en tant qu'Ahmed, le compte créé dans le secret `authelia_users`.

1.3 Bonus : Outline

1.3.a Les secrets d'Outline

Pour PostgreSQL, j'ai tiré le mot de passe une seule fois dans la variable `PW`. Il sert à 2 secrets et n'est jamais écrit sur le disque.

Commandes

```
PW=$(openssl rand -hex 24)
printf %s "$PW" | docker compose exec -T manager docker secret create outline_db_password -
printf %s "postgres://outline:$PW@postgres:5432/outline" | docker compose exec -T manager
↪  docker secret create outline_database_url -
for s in outline_secret_key outline_utils_secret; do openssl rand -hex 32 | tr -d "\n" |
↪  docker compose exec -T manager docker secret create $s -; done
openssl rand -hex 36 | tr -d "\n" > secrets/outline_oidc_secret
docker run --rm authelia/authelia:4.39 authelia crypto hash generate pbkdf2 --variant
↪  sha512 --password "$(cat secrets/outline_oidc_secret)"
docker compose exec -T manager docker secret create outline_oidc_secret - <
↪  secrets/outline_oidc_secret
```

1.3.b Un nouveau client dans Authelia

swarm/authelia.configuration.yml

```
# Bonus: Outline (TD4 ex. 1.3)
- client_id: 'outline'
  client_name: 'Outline'
  # ...
  redirect_uris:
    - 'https://outline.swarm.localhost/auth/oidc.callback'
```

Comme le rappelle l'énoncé, il faut supprimer puis recréer la config pour la mettre à jour. Cette fois, j'ai retiré la stack avant.

Commandes

```
docker compose exec manager docker stack rm authelia
docker compose exec manager docker config rm authelia_config
docker compose exec -T manager docker config create authelia_config - <
↪  authelia.configuration.yml
docker compose exec manager docker stack deploy -c /home/manager/authelia.stack.yml
↪  authelia
docker compose exec manager docker service ls --filter name=authelia
```



Figure 4.11. Comme la stack était déjà retirée, `config rm` a marché du premier coup.

1.3.c La stack

swarm/outline.stack.yml

```yaml
environment:
  SECRET_KEY_FILE: /run/secrets/outline_secret_key
  UTILS_SECRET_FILE: /run/secrets/outline_utils_secret
  DATABASE_URL_FILE: /run/secrets/outline_database_url
  # ...
  OIDC_CLIENT_ID: outline
  OIDC_CLIENT_SECRET_FILE: /run/secrets/outline_oidc_secret
  OIDC_AUTH_URI: https://authelia.swarm.localhost/api/oidc/authorization
  OIDC_TOKEN_URI: https://authelia.swarm.localhost/api/oidc/token
  OIDC_USERINFO_URI: https://authelia.swarm.localhost/api/oidc/userinfo
  # ...
  NODE_EXTRA_CA_CERTS: /certs/ca.crt
```

Commandes

```
docker compose cp outline.stack.yml manager:/home/manager/
docker compose exec manager docker stack deploy -c /home/manager/outline.stack.yml outline
docker compose exec manager docker stack services outline
docker compose exec manager docker service logs --raw outline_outline 2>&1 | tail -4
```

1.3.d Connexion à Outline

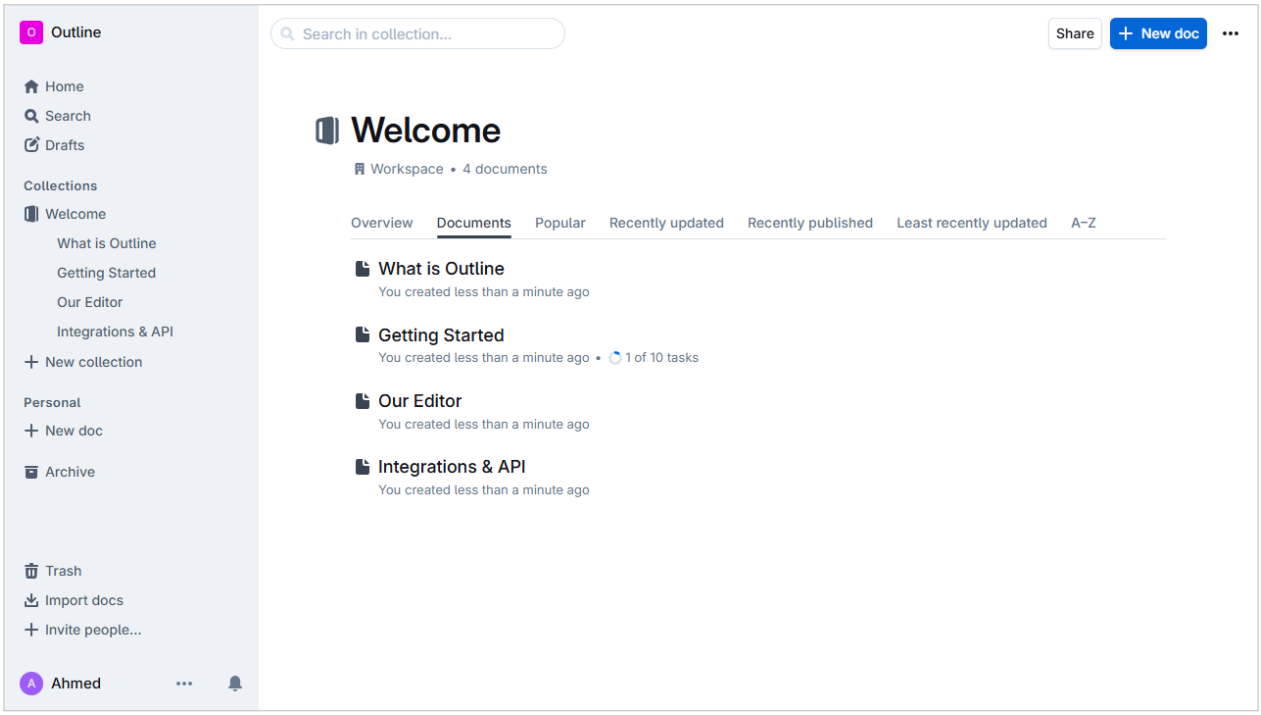


Figure 4.12. Après la connexion sur Authelia, Outline s'ouvre sur `outline.swarm.localhost`, connecté en tant qu'Ahmed.

Bilan

Nom	Type	Utilisé par	Créé depuis
`swarm_localhost_key`	secret	Traefik	ligne de commande
`swarm_localhost_crt`, `traefik_tls`	config	Traefik	ligne de commande
`swarm_ca`	config	Dozzle, Outline	ligne de commande
`dozzle_users`	secret	Dozzle (question 1.1)	Portainer
`dozzle_oidc_secret`	secret	Dozzle (question 1.2)	ligne de commande
`authelia_users`	secret	Authelia (le compte)	ligne de commande
`authelia_jwt_secret`, `authelia_session_secret`, `authelia_storage_key`, `authelia_oidc_hmac`, `authelia_oidc_key`	secret	Authelia	ligne de commande
`authelia_config`	config	Authelia	ligne de commande, recréée 2 fois
`outline_db_password`	secret	PostgreSQL d'Outline	ligne de commande
`outline_database_url`, `outline_secret_key`, `outline_utils_secret`, `outline_oidc_secret`	secret	Outline	ligne de commande

Au final, une config se relit et un secret jamais, mais aucun des deux ne se modifie.

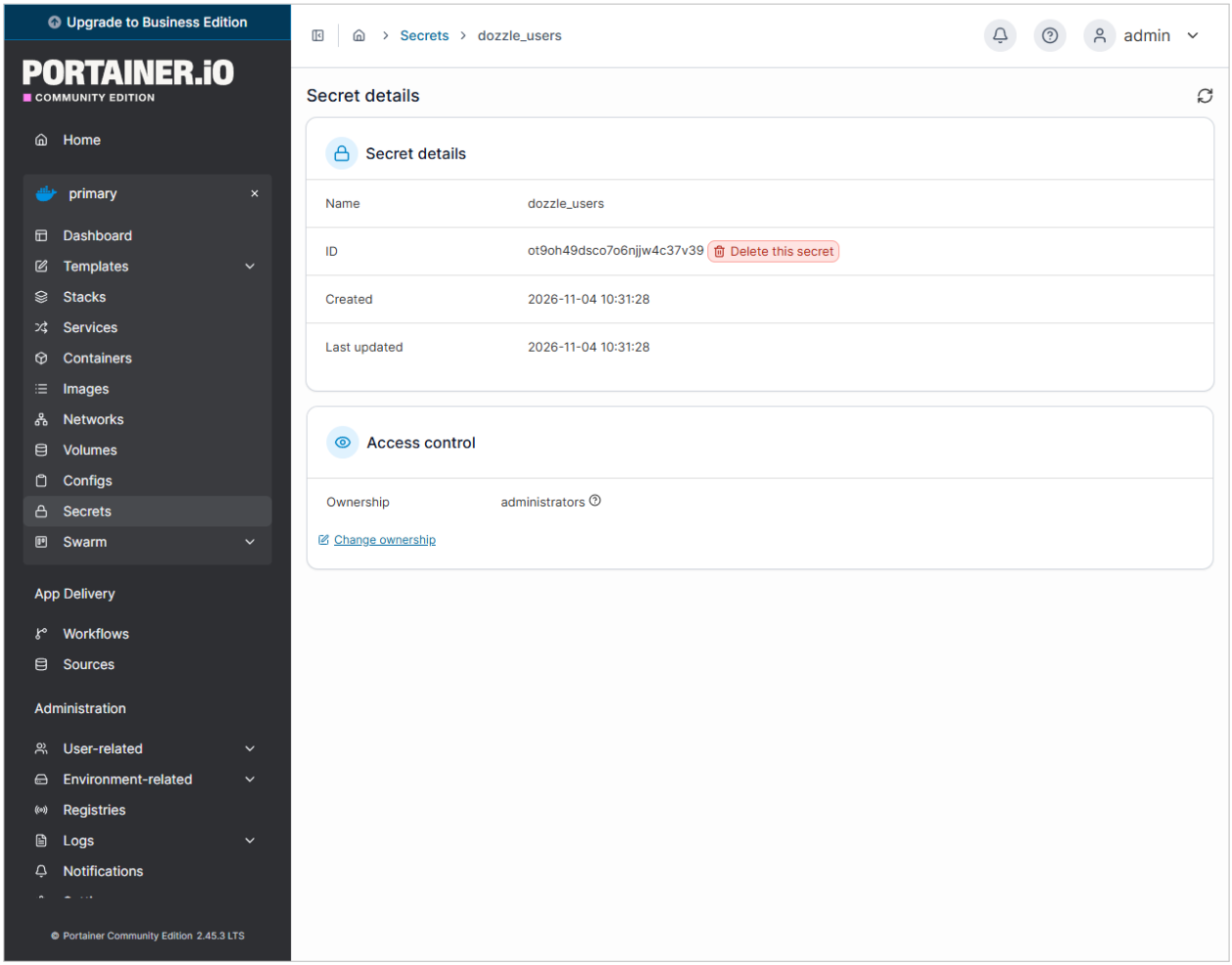


Figure 4.13. Dans Portainer, le secret `dozzle_users` n'affiche aucun contenu et ne peut être que supprimé.

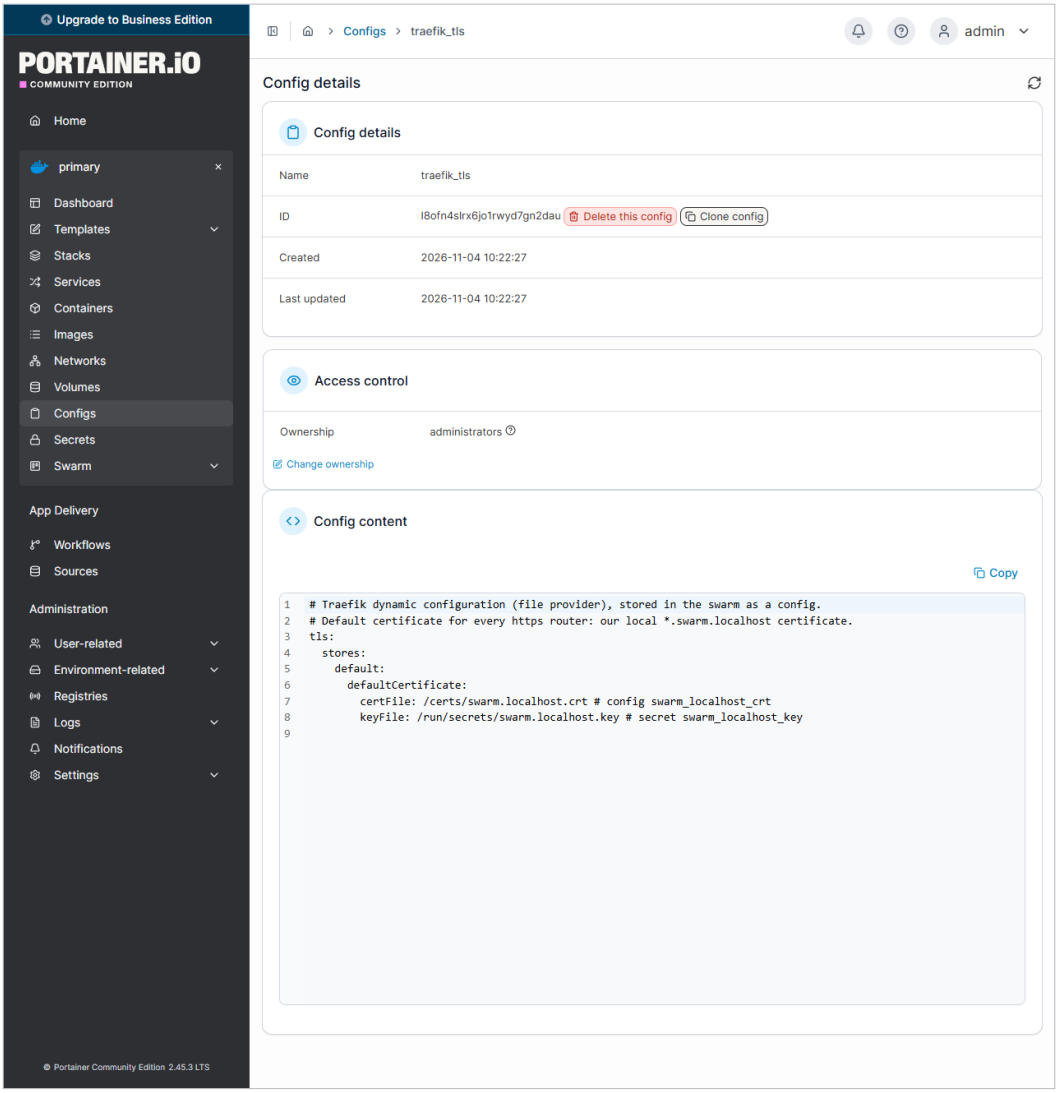


Figure 4.14. La config `traefik_tls`, elle, affiche son contenu, mais elle ne peut être que supprimée ou clonée.